

reglas_asoc_secretariado

Arlet Pinacho Báez Isaac Giovanni Escobar González
Kevin Jonathan Garduño Escobar Seldon Sautto Ramírez

2026-04-22

Table of contents

Introducción	4
Estructura del documento	4
1 Preprocesamiento del conjunto de datos	5
1.1 Introducción al conjunto de datos	5
1.2 Calidad de los datos	11
1.3 Discretización	13
1.4 Tratamiento de valores faltantes o confidenciales	15
2 Implementación manual del algoritmo Apriori	17
2.1 Algoritmo Apriori	17
2.1.1 Algoritmo 1: Generación de itemsets frecuentes	18
2.1.2 Explicación intuitiva	18
2.1.3 Propiedad Apriori (idea clave)	19
2.1.4 Algoritmo 2: Generación de reglas de asociación fuertes	19
2.1.5 Procedimiento	20
2.1.6 Explicación intuitiva	21
2.1.7 Relación entre ambos algoritmos	21
2.2 Implementación en Python	22
2.2.1 Definición de parámetros	22
2.2.2 Generación Manual de Itemsets Frecuentes	22
2.2.3 Generación Manual de Reglas de Asociación	26
2.3 Conclusión del Algoritmo Apriori Manual	35
3 Aplicación del algoritmo Apriori de python	38
3.1 Carga de datos	38
3.2 Definición de parámetros	38
Soporte mínimo	39
Confianza mínima	39
Tamaño máximo de los itemsets	39
3.3 Generación de itemsets frecuentes	40
3.4 Generación de Reglas de Asociación	41
3.5 Conclusión del Algoritmo Apriori de Python	49
4 Aplicación del algoritmo FP-Growth	50
4.1 Carga de datos	50

4.2	Definición de parámetros	51
4.2.1	Soporte mínimo	51
4.2.2	Confianza mínima	51
4.3	Generación de itemsets frecuentes	51
4.4	Generación de Reglas de Asociación	52
4.5	Conclusión de Algoritmo FP-Growth	55
5	Comparativa de los algoritmos	56
5.1	Comparativa de desempeño	56
	¿Cuál de ellos tarda más?	56
	¿Cuál tarda menos y por qué?	56
	¿Generan las mismas reglas de asociación? ¿Por qué?	56
6	Modelo de clasificación	57
6.1	Configuración del entorno y librerías	57
6.2	Carga de datos y exploración del dataset	58
6.3	Análisis de la Variable Objetivo	60
6.4	Análisis de las variables predictoras	62
6.5	Codificación One-Hot	64
6.6	División del conjunto de datos	68
6.7	Construcción del modelo	69
6.8	Predicciones	71
6.9	Métricas de Desempeño y Evaluación del Modelo	73
	6.9.1 Justificación de Métricas Seleccionadas	73
	6.9.2 Cálculo y Presentación de Métricas	74
6.10	Matriz de confusión	75
6.11	Análisis de Variables Relevantes para el Modelo de Clasificación	77
6.12	Patrones o Sesgos en los datos	80
	6.12.1 Patrones obtenidos	80
	6.12.2 Sesgos Detectados	80
6.13	Métricas y Relación con el mundo real	81
6.14	Visualización de un Árbol del Random Forest	83
6.15	Visualización de múltiples árboles del Random Forest	84
	References	86

Introducción

A lo largo de este proyecto se trabajará con el conjunto de datos *data_secretariado.csv* en una versión derivada del trabajo de limpieza realizado por otro equipo.

En primer lugar, se busca generar reglas de asociación interpretables, tomando en cuenta que estas **si bien pueden reflejar un fenómeno real también pueden ser consecuencia del mismo sistema de registro**. Durante el proceso de generación de reglas de asociación se aplicarán tres algoritmos distintos, con el fin de realizar una comparativa del desempeño de cada uno de ellos.

En segundo lugar, se busca desarrollar un modelo de clasificación, que sea capaz de predecir el valor de la variable `ESTATUS_VICTIMA` a partir de otras variables del dataset.

Estructura del documento

1 Preprocesamiento del conjunto de datos

En esta sección además de ver las características principales del conjunto de datos con el que se va a trabajar, se aplicarán transformaciones a los datos para discretizar las variables continuas presentes en el dataset, esto con el fin de mejorar el desempeño de los algoritmos de minería de datos que se aplicarán en las siguientes secciones.

1.1 Introducción al conjunto de datos

El conjunto de datos consta de **133,887 filas**, cada una representando un reporte de desaparición descrito por **11 variables** (columnas). Gran parte de estas columnas corresponden a datos representados mediante **cadenas de texto**, aunque también se tienen dos columnas con **datos numéricos**: CVE_ENT y CVE_MUN.

```
import pandas as pd

df = pd.read_csv("../data/data_secretariado.csv")

print(df.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 133887 entries, 0 to 133886
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ID_VICTIMA             133887 non-null  str
1   ORIGEN_REPORTE        133887 non-null  str
2   FECHA_NACIMIENTO      105000 non-null  str
3   SEXO                  133887 non-null  str
4   FECHA_DESAPARICION    125727 non-null  str
5   FECHA_REGISTRO        127790 non-null  str
6   ESTATUS_VICTIMA       133887 non-null  str
7   CVE_ENT               133887 non-null  int64
8   ENTIDAD               133887 non-null  str
9   CVE_MUN               133887 non-null  int64
```

```

10 MUNICIPIO          133887 non-null  str
dtypes: int64(2), str(9)
memory usage: 11.2 MB
None

```

Para clasificar de mejor forma las variables con las que se cuenta exploramos los posibles valores de cada columna.

```

for col in df.columns:
    print(col, df[col].unique())
    print()

```

```

ID_VICTIMA <StringArray>
['D49C001E-41E4-45B7-B8FD-D867578F093E',
 '2D2B5CD4-7AF7-48C3-887E-72F4C6403863',
 '6AE9098C-0C30-4709-9A2C-37F6312CD43E',
 '6B899D2D-F33D-4880-82CD-953FF972B9C3',
 '3F026638-8169-4CEA-9D9A-E345B6D2D7B6',
 '10F61F9C-4DE5-4A61-A977-B7F4EEE9E8AD',
 'BC0E6261-8D6E-4D31-B212-8AD9E73A63D6',
 'FD81E18D-216A-4104-8E10-BAC05608B419',
 '896BBA0C-4E84-49B2-AE4D-704D932D0399',
 'A86BB269-7295-4595-BBE7-F16C324D0D92',
 ...
 '03BD88B9-3425-4575-9280-9F09ACC668D4',
 '4FCC0509-82DF-4CDD-A5AC-F79DB9DF910B',
 '645AF4B0-17C0-4381-AB2A-B82399EEEB53',
 'CD6418E4-DB4C-4ABA-A955-4C0BC2A5D68F',
 '485256ED-DF26-468F-83F1-9572FAFBB80D',
 'E82861AE-C8AF-418F-ACA9-C215045C86B6',
 'C8C242CC-FE7E-4802-A24C-A4F038C22AF5',
 '3A125290-FBFA-4EEB-BB2D-5F4B19921FBC',
 'C714FA50-1CAA-483F-ABED-BA58883C499C',
 '1B66A897-3BB3-45DE-AB9D-49450DCFB545']
Length: 133887, dtype: str

```

```

ORIGEN_REPORTE <StringArray>
[
    'FISCALIA GENERAL DE DURANGO',
    'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE SINALOA',
    'PROCURADURIA GENERAL DE JUSTICIA DE LA CIUDAD DE MEXICO',
    'FISCALIA GENERAL DEL ESTADO BAJA CALIFORNIA',
    'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE PUEBLA',

```

'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE MICHOACAN',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE BAJA CALIFORNIA',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE GUANAJUATO',
 'PROCURADURIA GENERAL DE JUSTICIA DEL ESTADO DE HIDALGO',
 'FISCALIA GENERAL DE MORELOS',
 'PROCURADURIA GENERAL DE JUSTICIA DEL ESTADO DE TLAXCALA',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DE LA CIUDAD DE MEXICO',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE NAYARIT',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE NUEVO LEON',
 'COMISION NACIONAL DE BUSQUEDA DE PERSONAS',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE HIDALGO',
 'FISCALIA GENERAL DE SONORA',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE AGUASCALIENTES',
 'PROCURADURIA GENERAL DE JUSTICIA DEL ESTADO DE MICHOACAN',
 'FISCALIA GENERAL DE PUEBLA',
 'FISCALIA GENERAL DE NAYARIT',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE CAMPECHE',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE CHIHUAHUA',
 'FISCALIA GENERAL DEL ESTADO DE CAMPECHE',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE QUERETARO',
 'FISCALIA GENERAL DE GUERRERO',
 'FISCALIA GENERAL DE ESTADO DE MEXICO',
 'PROCURADURIA GENERAL DE JUSTICIA DEL ESTADO DE TAMAULIPAS',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE BAJA CALIFORNIA SUR',
 'FISCALIA GENERAL DE SINALOA',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE OAXACA',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE YUCATAN',
 'FISCALIA GENERAL DE ZACATECAS',
 'FISCALIA GENERAL DE TABASCO',
 'FISCALIA GENERAL DE JALISCO',
 'FISCALIA GENERAL DE CHIAPAS',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE TLAXCALA',
 'FISCALIA GENERAL DE CHIHUAHUA',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE ZACATECAS',
 'FISCALIA GENERAL DE VERACRUZ',
 'FISCALIA GENERAL DE NUEVO LEON',
 'PROCURADURIA GENERAL DE JUSTICIA DEL ESTADO DE BAJA CALIFORNIA SUR',
 'FISCALIA GENERAL DEL ESTADO DE AGUASCALIENTES',
 'FISCALIA GENERAL DE QUINTANA ROO',
 'FISCALIA GENERAL DE QUERETARO',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE MEXICO',
 'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE COAHUILA',
 'PORTAL',

```

'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE SAN LUIS POTOSI',
'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE COLIMA',
'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE MORELOS',
'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE SONORA',
'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE CHIAPAS',
'PROCURADURIA GENERAL DE JUSTICIA DEL ESTADO DE GUANAJUATO',
'FISCALIA GENERAL DE LA REPUBLICA',
'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE QUINTANA ROO',
'FISCALIA GENERAL DE SAN LUIS POTOSI',
'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE VERACRUZ',
'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE DURANGO',
'FISCALIA GENERAL DE YUCATAN',
'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE TABASCO',
'FISCALIA GENERAL DE OAXACA',
'FISCALIA GENERAL DEL ESTADO DE COAHUILA',
'FISCALIA GENERAL DEL ESTADO DE COLIMA',
'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE TAMAULIPAS',
'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE JALISCO',
'COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE GUERRERO',
'SE DESCONOCE',
'SUBPROCURADURIA DE DERECHOS HUMANOS, PREVENCION DEL DELITO Y SERVICIOS A LA COMUNIDAD',
'FISCALIA ESPECIAL PARA LOS DELITOS DE VIOLENCIA CONTRA LAS MUJERES Y TRATA DE PERSONAS',
'SUBPROCURADURIA ESPECIALIZADA EN INVESTIGACION DE DELINCUENCIA ORGANIZADA',
'SUBPROCURADURIA ESPECIALIZADA EN INVESTIGACION DE DELITOS FEDERALES',
'SUBPROCURADURIA DE CONTROL REGIONAL, PROCEDIMIENTOS PENALES Y AMPARO',
'SUBPROCURADURIA DERECHOS HUMANOS, ATENCION A VICTIMAS Y SERVICIOS A LA COMUNIDAD']
Length: 74, dtype: str

```

```

FECHA_NACIMIENTO <StringArray>
['CONFIDENCIAL', '1971-06-13', '2022-12-17', '1992-09-16',
'1990-06-26', '2021-05-10', '1982-08-22', '1980-08-14',
'1996-01-28', '1978-04-17',
...
'1940-02-08', '1952-05-18', '1956-08-22', '1950-10-28',
'1935-03-12', '1964-01-26', '1975-11-03', '1944-06-12',
'1930-05-01', '1945-06-24']
Length: 20809, dtype: str

```

```

SEXO <StringArray>
['CONFIDENCIAL', 'HOMBRE', 'MUJER', 'INDETERMINADO']
Length: 4, dtype: str

```

```

FECHA_DESAPARICION <StringArray>

```



```
[
    'CONFIDENCIAL', '2025-09-28 04:40:00', '2025-09-20 13:00:00',
    '2021-03-16 07:00:00', '2025-09-29 12:00:00', '2025-05-26 06:00:00',
    '2025-09-19 19:30:00', '2025-09-28 21:40:00', '2025-09-27 10:00:00',
    '2025-09-28 15:00:00',
    ...
    '2006-12-20 01:30:00', '2006-11-26 12:00:00', '2006-12-11 05:00:00',
    '2006-12-14 07:09:09', '2006-12-10 09:00:00', '2006-11-28 10:58:00',
    '2006-12-02 06:00:00', '2006-12-03 12:00:00', '2006-12-01 04:30:00',
    '2006-12-01 06:30:00']
Length: 50384, dtype: str
```

```
FECHA_REGISTRO <StringArray>
[
    'CONFIDENCIAL', '2025-09-29 10:00:00', '2025-09-20 13:00:00',
    '2021-03-16 20:00:00', '2025-10-01 15:18:00', '2025-05-26 06:00:00',
    '2025-09-19 19:30:00', '2025-10-01 14:59:00', '2025-09-27 10:00:00',
    '2025-09-28 15:00:00',
    ...
    '2006-12-20 01:30:00', '2006-11-26 12:00:00', '2006-12-11 05:00:00',
    '2006-12-14 07:09:09', '2006-12-10 09:00:00', '2006-11-28 10:58:00',
    '2006-12-02 06:00:00', '2006-12-03 12:00:00', '2006-12-01 04:30:00',
    '2006-12-01 06:30:00']
Length: 53636, dtype: str
```

```
ESTATUS_VICTIMA <StringArray>
['CONFIDENCIAL', 'DESAPARECIDA', 'NO LOCALIZADA']
Length: 3, dtype: str
```

```
CVE_ENT [10 25  9  2 21 16 11 13 17 29 18 19 26  1  4  8 22 20 12 15 28  3 31 32
         27 14  7 30 23  6  5 99 24]
```

```
ENTIDAD <StringArray>
[
    'DURANGO',          'SINALOA',          'CIUDAD DE MÉXICO',
    'BAJA CALIFORNIA',  'PUEBLA',          'MICHOACÁN',
    'GUANAJUATO',       'HIDALGO',          'MORELOS',
    'TLAXCALA',         'NAYARIT',          'NUEVO LEÓN',
    'SONORA',           'AGUASCALIENTES',   'CAMPECHE',
    'CHIHUAHUA',        'QUERÉTARO',        'OAXACA',
    'GUERRERO',         'ESTADO DE MÉXICO', 'TAMAULIPAS',
    'BAJA CALIFORNIA SUR', 'YUCATÁN',          'ZACATECAS',
    'TABASCO',          'JALISCO',          'CHIAPAS',
    'VERACRUZ',         'QUINTANA ROO',      'COLIMA',
    'COAHUILA',         'SE DESCONOCE',     'SAN LUIS POTOSÍ']
Length: 33, dtype: str
```

```

CVE_MUN [999  4 114  91  48  6 17  5 75 43 53 71 66 202 47 125 174  2
 23 39 12 30 18  7 16  1 143 27 32 35  8 157 19 58 87 56
 40 74 25 61 34 14 41 76 44 77 51 97 33 10 11 69 55 176
 68 113 37 59  9 78 108 49 177 208 118 29 31 57 70  3 191 95
109 90 50 101 13 38 22 193 60 93 140 21 45 84 52 131 175 111
120 121 24 104 154 85 124 15 189 156 42 92 551 86 89 106 28 26
 20 63 62 99 36 46 482 318 179 132 112 197 88 81 194 67 82 119
164 207 167 122 54 80 65 103 83 102 418 110 136 96 123 79 145 115
149 126 107 401 399 141 98 170 216 72 94 105 73 116 134 188 213 160
205 155 161 100 64 310 555 337 203 184 169 151 212 192 270 172 413 391
570 243 394 204 144 378 515 142 185 181 415 148 180 163 138 385 498 334
509 159 158 476 182 130 563 211 173 390 187 546 565 296 127 324 375 117
153 319 133 371 168 166 340 186 190 294 199 200 128 409 178 348 333 214
247 469 198 196 209 210 364 397 277 545 421 150 439 232 206 494 152 183
483 554 473 350 406 295 525 386 139 519 147 523 414 201 135 165 146 195
129 558 447 217 264]

```

```

MUNICIPIO <StringArray>
[
    'CONFIDENCIAL',
    'PUEBLA',
    'PACHUCA DE SOTO',
    'TEPIC',
    'TOLCAYUCA',
    ...
    'EJUTLA',
    'NANACAMILPA DE MARIANO ARISTA',
    'VIESCA',
    'CAÑADAS DE OBREGÓN',
    'ONAVAS',
    'CONCORDIA',
    'TINGÜINDÍN',
    'CALPULALPAN',
    'AJACUBA',
    'YAUHQUEMECAN',
    'JILOTLÁN DE LOS DOLORES',
    'MAINERO',
    'SANTA MARÍA DE LOS ÁNGELES',
    'TOTATICHE',
    'TUBUTAMA']
Length: 1554, dtype: str

```

Así, la siguiente tabla resume el tipo de información que brinda cada columna del dataset.

Columna	Significado	Tipo de variable
ID_VICTIMA	Folio Único de Búsqueda (tentativamente)	Cualitativa nominal
ORIGEN_REPORTE	Institución que generó el reporte	Cualitativa nominal
FECHA_NACIMIENTO	Fecha de nacimiento de la víctima	Numérica continua

Columna	Significado	Tipo de variable
SEXO	Sexo de la víctima	Cualitativa nominal
FECHA_DE-SAPARICION	Fecha y hora en que ocurrió la desaparición	N Numérica continua
FECHA_REG-ISTRO	Fecha y hora en que se registró el caso	N Numérica continua
ESTA-TUS_VIC-TIMA	Estado actual de la víctima	C Cualitativa nominal
CVE_ENT	Clave de la entidad federativa	C Cualitativa nominal
ENTIDAD	Nombre de la entidad federativa	C Cualitativa nominal
CVE_MUN	Clave del municipio	C Cualitativa nominal
MUNICIPIO	Nombre del municipio	C Cualitativa nominal

Por último, para facilitar el entendimiento de los registros del conjunto de datos, veremos algunas de las filas contenidas.

```
df.head(5)
```

	ID_VICTIMA	ORIGEN_REPORTE
0	D49C001E-41E4-45B7-B8FD-D867578F093E	FISCALIA GENERAL DE DURANGO
1	2D2B5CD4-7AF7-48C3-887E-72F4C6403863	COMISION LOCAL DE BUSQUEDA DE PERSONAS DE
2	6AE9098C-0C30-4709-9A2C-37F6312CD43E	PROCURADURIA GENERAL DE JUSTICIA DE LA CIU
3	6B899D2D-F33D-4880-82CD-953FF972B9C3	FISCALIA GENERAL DEL ESTADO BAJA CALIFORNI
4	3F026638-8169-4CEA-9D9A-E345B6D2D7B6	FISCALIA GENERAL DEL ESTADO BAJA CALIFORNI

1.2 Calidad de los datos

En la siguiente gráfica es posible comparar la cantidad de datos confidenciales, faltantes y desconocidos de cada una de las columnas de interés del conjunto de datos.

```

import plotly.express as px

cols_interes = ['ORIGEN_REPORTE', 'FECHA_NACIMIENTO', 'SEXO', 'FECHA_DESAPARICION', 'FECHA_REGISTRO']

res = []

for col in cols_interes:
    total = len(df)

    # Conteo de nulos, confidenciales y validos
    nulos = df[col].isnull().sum()
    confidenciales = (df[col] == "CONFIDENCIAL").sum()
    desconocidos = (df[col] == "SE DESCONOCE").sum()
    validos = total - nulos - confidenciales - desconocidos

    # Guardamos proporciones
    for tipo, conteo in [('Faltante', nulos), ('Confidencial', confidenciales), ('Desconocido', desconocidos)]:
        res.append({
            'Columna': col,
            'Estado': tipo,
            'Porcentaje': (conteo / total) * 100
        })

df_plot = pd.DataFrame(res)

# Gráfica de barras apiladas
fig = px.bar(
    df_plot,
    x='Columna',
    y='Porcentaje',
    color='Estado',
    text='Porcentaje',
    color_discrete_map={
        'Faltante': '#821300',
        'Confidencial': '#015dfc',
        'Válido': '#2e9302'
    }
)

fig.update_traces(
    texttemplate='%{text:.1f}%',
    textposition='inside'
)

```

```

)

fig.update_layout(
    yaxis_title='Porcentaje del total (%)',
    xaxis_title='Columna',
    legend_title='Estado del dato',
    barmode='stack'
)

fig.show()

```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

1.3 Discretización

Como ya vimos en el punto anterior el dataset cuenta con tres columnas con datos numéricos continuos las que serán las variables que necesitan discretización. Estas columnas son **FECHA_NACIMIENTO**, **FECHA_DESAPARICION** y **FECHA_REGISTRO**.

Para empezar a discretizar estas variables, se decidió lo siguiente: **FECHA_NACIMIENTO** se discretizará en rangos de edad, es decir, calcularemos la edad que tenía la víctima al momento de su desaparición y luego se agruparán en rangos como: 0-11 años, 12-17 años, 18-29 años, 30-44 años, 45-59 años y 60 años o más.

Al elegir estos rangos nos basamos en que cada uno representa una etapa de la vida como: infancia, adolescencia, juventud, adultez temprana, adultez media y vejez, lo que puede ser relevante en el análisis de desapariciones, además de que son rangos muy parecidos a los que se usan en los informes de Personas Desaparecidas de Datacívica. En cuanto a las columnas **FECHA_DESAPARICION** y **FECHA_REGISTRO**, se decidió discretizarlas de acuerdo a los meses del año en que ocurrieron, es decir, se agruparán en categorías como: enero, febrero, marzo, abril, mayo, junio, julio, agosto, septiembre, octubre, noviembre y diciembre, además de por año en el que ocurrió. Esta elección se hizo así para poder analizar de manera más fácil si existen algún patrón en las desapariciones o en los registros de acuerdo a la época del año o al año mismo en el que ocurrieron.

Por lo tanto, se crearán nuevas columnas para cada una de estas variables discretizadas, quedando de la siguiente manera: **RANGO_EDAD**, **MES_DESAPARICION**, **AÑO_DESAPARICION**, **MES_REGISTRO** y **AÑO_REGISTRO**, además de las variables que no necesitan discretización puesto que ya son categóricas.

Por otro lado, decidimos eliminar las columnas: **ID_VICTIMA**, **FECHA_NACIMIENTO**, **FECHA_DESAPARICION**, **FECHA_REGISTRO**, **CVE_ENT** y **CVE_MUN** ya que conservar estas

columnas no aportaría información relevante, además de que las columnas CVE_ENT y CVE_MUN son claves numéricas de las entidades y los municipios lo cual no es necesario para el análisis.

```
# Calcular edad al momento de la desaparición
df['FECHA_NACIMIENTO'] = pd.to_datetime(df['FECHA_NACIMIENTO'], format='%Y-%m-%d', errors='coerce')
df['FECHA_DESAPARICION'] = pd.to_datetime(df['FECHA_DESAPARICION'], format='%Y-%m-%d %H:%M:%S', errors='coerce')
df['FECHA_REGISTRO'] = pd.to_datetime(df['FECHA_REGISTRO'], format='%Y-%m-%d %H:%M:%S', errors='coerce')

df['EDAD'] = (df['FECHA_DESAPARICION'] - df['FECHA_NACIMIENTO']).dt.days // 365

# Agrupar rangos de edad
bins = [0, 12, 18, 30, 45, 60, 120]
labels = ['0-11', '12-17', '18-29', '30-44', '45-59', '60+']
df['GRUPO_EDAD'] = pd.cut(df['EDAD'], bins=bins, labels=labels, right=False).astype('object')

# Mes y año de desaparición
df['MES_DESAPARICION'] = df['FECHA_DESAPARICION'].dt.month
df['AÑO_DESAPARICION'] = df['FECHA_DESAPARICION'].dt.year

# Mes y año de registro
df['MES_REGISTRO'] = df['FECHA_REGISTRO'].dt.month
df['AÑO_REGISTRO'] = df['FECHA_REGISTRO'].dt.year

# Mes numérico a nombre
meses = {1:'Enero',2:'Febrero',3:'Marzo',4:'Abril',5:'Mayo',6:'Junio',
        7:'Julio',8:'Agosto',9:'Septiembre',10:'Octubre',11:'Noviembre',12:'Diciembre'}

df['MES_DESAPARICION'] = df['MES_DESAPARICION'].map(meses)
df['MES_REGISTRO'] = df['MES_REGISTRO'].map(meses)

# En caso de que no se haya podido calcular la edad, se asigna el valor 'DESCONOCIDO' al grupo
df['GRUPO_EDAD'] = df['GRUPO_EDAD'].astype('object')
df['GRUPO_EDAD'] = df['GRUPO_EDAD'].fillna('DESCONOCIDO')

print("\nDistribución de GRUPO_EDAD:")
print(df['GRUPO_EDAD'].value_counts())
```

Distribución de GRUPO_EDAD:

GRUPO_EDAD	
DESCONOCIDO	82989
18-29	18291

30-44	17073
45-59	6316
12-17	4473
60+	2653
0-11	2092

Name: count, dtype: int64

1.4 Tratamiento de valores faltantes o confidenciales

Decidimos no eliminar los registros con valores faltantes o confidenciales, ya que aunque no tengamos la información completa en estos registros eso no significa que no tengamos nada de información en estos registros, además de que eliminar estos registros podría reducir significativamente el tamaño del conjunto de datos, perder información valiosa y alterar el análisis de los datos.

En lugar de eliminar estos registros decidimos crear una nueva categoría DESCONOCIDO para unificar los valores faltantes o confidenciales, esto para que el análisis de los datos sea más fácil y para que no se pierda información valiosa.

Entonces, se reemplazaron los valores CONFIDENCIAL, SIN DATO, INDETERMINADO y SE DESCONOCE por la categoría DESCONOCIDO en todas las columnas del dataset.

```
valores_confidenciales = ['CONFIDENCIAL', 'SIN DATO', 'INDETERMINADO', 'SE DESCONOCE']
for col in df.select_dtypes(include=['object', 'str']).columns:
    n = df[col].isin(valores_confidenciales).sum()
    if n > 0:
        print(f"Columna '{col}': {n} registros ({100*n/len(df):.1f}%) reemplazados por 'DESCONOCIDO'")
        df.loc[df[col].isin(valores_confidenciales), col] = 'DESCONOCIDO'

for col in ['GRUPO_EDAD', 'MES_DESAPARICION', 'AÑO_DESAPARICION', 'MES_REGISTRO', 'AÑO_REGISTRO']:
    df[col] = df[col].fillna('DESCONOCIDO')
```

Columna 'ORIGEN_REPORTE': 574 registros (0.4%) reemplazados por 'DESCONOCIDO'

Columna 'SEXO': 49505 registros (37.0%) reemplazados por 'DESCONOCIDO'

Columna 'ESTATUS_VICTIMA': 49149 registros (36.7%) reemplazados por 'DESCONOCIDO'

Columna 'ENTIDAD': 2953 registros (2.2%) reemplazados por 'DESCONOCIDO'

Columna 'MUNICIPIO': 54545 registros (40.7%) reemplazados por 'DESCONOCIDO'

Ahora, se eliminaron las columnas que ya no se necesitan para el análisis, de modo que se guarda un nuevo dataset con las siguientes columnas: ID_VICTIMA, ORIGEN_REPORTE, SEXO, ESTATUS_VICTIMA, ENTIDAD, MUNICIPIO, GRUPO_EDAD, MES_DESAPARICION, AÑO_DESAPARICION, MES_REGISTRO y AÑO_REGISTRO.

```
columnas_analisis = ['ORIGEN_REPORTE', 'SEXO', 'ESTATUS_VICTIMA', 'ENTIDAD', 'MUNICIPIO',
                    'GRUPO_EDAD', 'MES_DESAPARICION', 'AÑO_DESAPARICION', 'MES_REGISTRO', 'AÑO_REGISTRO']
df_analisis = df[columnas_analisis].copy()
df_analisis.head(5)
```

	ORIGEN_REPORTE	SEXO	ESTATUS_VIC
0	FISCALIA GENERAL DE DURANGO	DESCONOCIDO	DESCONOCIDO
1	COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL EST...	HOMBRE	DESAPARECIDO
2	PROCURADURIA GENERAL DE JUSTICIA DE LA CIUDAD ...	DESCONOCIDO	DESCONOCIDO
3	FISCALIA GENERAL DEL ESTADO BAJA CALIFORNIA	DESCONOCIDO	DESCONOCIDO
4	FISCALIA GENERAL DEL ESTADO BAJA CALIFORNIA	DESCONOCIDO	DESCONOCIDO

Finalmente, se guarda un nuevo dataset listo para Apriori y clasificación, los cuales son: `data_secretariado_analisis.csv` y `data_secretariado_clasificacion.csv`

Como nota, hay que mencionar que las columnas `AÑO_DESAPARICION` y `AÑO_REGISTRO` las convertimos a tipo `str` para facilitar el análisis de los datos y para que no se pierda información valiosa.

```
df_analisis['AÑO_DESAPARICION'] = df_analisis['AÑO_DESAPARICION'].astype(str)
df_analisis['AÑO_REGISTRO'] = df_analisis['AÑO_REGISTRO'].astype(str)

df_apriori = df_analisis.copy()
df_apriori = pd.get_dummies(df_apriori, dtype=bool)
df_apriori.to_csv("../data/data_secretariado_apriori.csv", index=False)
print("Dataset preprocesado guardado como 'data_secretariado_apriori.csv'")
print(f"Dataset guardado: {df_apriori.shape[0]} filas, {df_apriori.shape[1]} columnas")

df_analisis.to_csv("../data/data_secretariado_clasificacion.csv", index=False)
print("Dataset preprocesado guardado como 'data_secretariado_clasificacion.csv'")
print(f"Dataset guardado: {df_analisis.shape[0]} filas, {df_analisis.shape[1]} columnas")
```

```
Dataset preprocesado guardado como 'data_secretariado_apriori.csv'
Dataset guardado: 133887 filas, 1825 columnas
Dataset preprocesado guardado como 'data_secretariado_clasificacion.csv'
Dataset guardado: 133887 filas, 10 columnas
```


2 Implementación manual del algoritmo Apriori

El algoritmo **Apriori** es un método de minería de datos utilizado para descubrir patrones frecuentes dentro de una base de datos transaccional. Su objetivo principal es encontrar combinaciones de ítems que aparecen juntas muchas veces dentro del conjunto de datos.

Apriori funciona de manera **iterativa y por niveles**. Primero encuentra los ítems frecuentes individuales (de tamaño 1), después forma combinaciones de tamaño 2, luego de tamaño 3, y así sucesivamente. En cada nivel solo se conservan aquellas combinaciones que cumplen con un **soporte mínimo** previamente definido.

La idea central del algoritmo se basa en la **propiedad Apriori**, que dice:

Si un conjunto de ítems es frecuente, entonces todos sus subconjuntos no vacíos también deben ser frecuentes.

Gracias a esta propiedad, el algoritmo evita evaluar muchas combinaciones innecesarias. Si un conjunto pequeño no es frecuente, el algoritmo descarta automáticamente cualquier combinación más grande que lo contenga. Esto hace que Apriori sea más eficiente que un enfoque completamente exhaustivo.

Es importante mencionar que Apriori, por sí mismo, se encarga de generar los **itemsets frecuentes**. Posteriormente, a partir de estos conjuntos frecuentes, se construyen las **reglas de asociación fuertes**, utilizando un umbral mínimo de confianza.

En esta sección se desarrolla una implementación manual del algoritmo en Python, siguiendo el pseudocódigo visto en clase y replicando paso a paso la lógica del procedimiento.

2.1 Algoritmo Apriori

El algoritmo Apriori se compone de dos procedimientos principales:

1. Generación de **itemsets frecuentes**.
2. Generación de **reglas de asociación fuertes** a partir de esos itemsets.

A continuación se describe el primer procedimiento siguiendo el pseudocódigo visto en clase e incorporando una explicación intuitiva.

2.1.1 Algoritmo 1: Generación de itemsets frecuentes

Entrada:

- Una base de datos transaccional D .
- Un valor mínimo de soporte.

Salida:

- El conjunto de todos los itemsets frecuentes en D .

El procedimiento funciona de la siguiente manera:

1. Se inicia con $k = 1$.
 2. Se generan los candidatos de tamaño k (conjuntos de un solo ítem).
 3. Se calcula el soporte de cada candidato recorriendo la base de datos.
 4. Se construye F_k , que contiene únicamente los candidatos que cumplen el soporte mínimo.
 5. Mientras $F_k \neq \emptyset$:
 - Se incrementa el tamaño: $k = k + 1$.
 - Se generan nuevos candidatos C_k combinando los conjuntos frecuentes del nivel anterior.
 - Se calcula el soporte de cada candidato recorriendo nuevamente la base de datos.
 - Se eliminan los candidatos que no cumplen el soporte mínimo.
 6. El algoritmo termina cuando ya no se generan nuevos conjuntos frecuentes.
 7. Se devuelve la unión de todos los conjuntos frecuentes encontrados.
-

2.1.2 Explicación intuitiva

Apriori trabaja **nivel por nivel**.

Primero busca qué elementos individuales aparecen muchas veces (nivel 1).

Después combina esos elementos frecuentes para formar pares (nivel 2).

Luego combina los pares frecuentes para formar tríos (nivel 3), y así sucesivamente.

En cada nivel:

- Se generan candidatos.
 - Para cada candidato se recorre la base de datos completa.
 - Se cuenta cuántas transacciones contienen ese conjunto.
 - Se calcula su soporte.
 - Solo sobreviven los que cumplen el soporte mínimo.
-

2.1.3 Propiedad Apriori (idea clave)

El algoritmo se basa en la siguiente propiedad:

Si un conjunto de ítems es frecuente, entonces todos sus subconjuntos no vacíos también deben ser frecuentes.

Esto significa que si un subconjunto no cumple el soporte mínimo, el algoritmo **ni siquiera genera** combinaciones más grandes que lo contengan.

Por ejemplo, si el conjunto $\{A, C\}$ no es frecuente, entonces $\{A, B, C\}$ no puede ser frecuente. Por lo tanto, esa combinación no se construye en el siguiente nivel.

Gracias a esta propiedad, el algoritmo evita evaluar muchas combinaciones innecesarias y reduce significativamente la cantidad de cálculos.

2.1.4 Algoritmo 2: Generación de reglas de asociación fuertes

Una vez que se han identificado los itemsets frecuentes mediante el Algoritmo 1, el siguiente paso consiste en generar reglas de asociación a partir de dichos conjuntos.

Mientras que el primer algoritmo responde a la pregunta:

¿Qué combinaciones de elementos aparecen con frecuencia?

El segundo responde:

Si ocurre un conjunto de elementos, ¿qué tan probable es que también ocurra otro?

Entrada:

- El conjunto de itemsets frecuentes.
- Un valor mínimo de confianza c_{min} .

Salida:

- El conjunto R de reglas de asociación fuertes.
-

2.1.5 Procedimiento

1. Se inicializa el conjunto de reglas como vacío:

$$R = \emptyset$$

2. Para cada itemset frecuente I de tamaño $k \geq 2$:

- Se generan todos los subconjuntos no vacíos $s \subset I$.
- Para cada subconjunto s , se construye la regla:

$$s \Rightarrow (I - s)$$

3. Para cada regla generada, se calcula su confianza:

$$confidence(s \Rightarrow I - s) = \frac{support(I)}{support(s)}$$

4. Si la confianza es mayor o igual que c_{min} , la regla se conserva.
 5. Si no cumple el umbral mínimo, la regla se descarta.
 6. El proceso se repite para todos los itemsets frecuentes.
 7. Finalmente, se devuelve el conjunto de reglas fuertes R .
-

2.1.6 Explicación intuitiva

Si un conjunto frecuente es:

$$I = \{A, B, C\}$$

El algoritmo intenta convertirlo en reglas posibles:

- $A \Rightarrow BC$
- $B \Rightarrow AC$
- $C \Rightarrow AB$
- $AB \Rightarrow C$
- $AC \Rightarrow B$
- $BC \Rightarrow A$

Cada una de estas reglas se evalúa mediante la confianza.

La confianza mide la probabilidad condicional:

¿Qué tan probable es que ocurra el consecuente si ya ocurrió el antecedente?

Si una regla tiene alta confianza, significa que cuando aparece el antecedente, el consecuente aparece con gran frecuencia.

2.1.7 Relación entre ambos algoritmos

El Algoritmo 1 identifica patrones frecuentes utilizando soporte.

El Algoritmo 2 toma esos patrones y construye implicaciones utilizando confianza.

Es importante notar que:

- No se generan reglas a partir de conjuntos que no sean frecuentes.
- El soporte garantiza relevancia estadística.
- La confianza garantiza fuerza predictiva.

Ambos procedimientos trabajan de manera complementaria para descubrir patrones estructurales dentro del conjunto de datos.

Primero se detectan combinaciones relevantes; posteriormente, se identifican relaciones con alta probabilidad condicional.

2.2 Implementación en Python

En esta sección se implementa manualmente la primera parte del algoritmo Apriori, es decir, la generación de itemsets frecuentes, siguiendo el pseudocódigo del Algoritmo 4.1.

El objetivo es:

- Generar candidatos por niveles.
- Calcular su soporte recorriendo la base de datos.
- Conservar únicamente aquellos que cumplan el soporte mínimo.

2.2.1 Definición de parámetros

Soporte mínimo

Se define un soporte mínimo de **10% (0.1)**.

Justificación: El dataset contiene 133,887 registros por lo que un soporte de 10% implica que un patrón debe aparecer en al menos 13,388 casos para considerarse relevante. Un valor más alto podría limitar la generación de reglas debido al desbalance en las categorías que presenta el conjunto de datos original.

Confianza mínima

Se define una confianza mínima de **70% (0.7)**.

Justificación:

- Una confianza de 70% garantiza que, dado el antecedente, el consecuente ocurre con alta probabilidad.
- Permite descartar reglas débiles o poco consistentes.
- Aumenta la interpretabilidad de resultados.

2.2.2 Generación Manual de Itemsets Frecuentes

```
import pandas as pd
import time
from itertools import combinations

# =====
# Cargar datos
```

```

# =====

df = pd.read_csv("../data/data_secretariado_apriori.csv")

# Convertimos cada fila en un conjunto de ítems verdaderos
transactions = []

for _, row in df.iterrows():
    items = set(df.columns[row == 1])
    transactions.append(items)

n_transactions = len(transactions)

print("Total transacciones:", n_transactions)

# =====
# Parámetro
# =====

min_support = 0.1
min_count = min_support * n_transactions

print("Soporte mínimo en cantidad:", min_count)

# =====
# Función para calcular soporte
# =====

def calculate_support(itemset):
    count = 0
    for transaction in transactions:
        if itemset.issubset(transaction):
            count += 1
    return count

# =====
# Algoritmo Apriori Manual
# =====

k = 1
frequent_itemsets = []
current_frequent = []

```

```

# ----- Nivel 1 -----
print("\nGenerando F1...")

item_counts = {}

inicio = time.time()

for transaction in transactions:
    for item in transaction:
        item_counts[item] = item_counts.get(item, 0) + 1

F1 = []

for item, count in item_counts.items():
    if count >= min_count:
        F1.append(frozenset([item]))

current_frequent = F1
frequent_itemsets.extend(F1)

print("Itemsets frecuentes tamaño 1:", len(F1))

# ----- Niveles k >= 2 -----

while current_frequent:

    k += 1
    print(f"\nGenerando candidatos tamaño {k}...")

    candidates = set()

    # Generar Ck combinando Fk-1
    for i in range(len(current_frequent)):
        for j in range(i + 1, len(current_frequent)):
            union = current_frequent[i].union(current_frequent[j])
            if len(union) == k:
                candidates.add(union)

    print("Candidatos generados:", len(candidates))

    # Calcular soporte
    new_frequent = []

```



```

    for candidate in candidates:
        count = calculate_support(candidate)
        if count >= min_count:
            new_frequent.append(candidate)

    print("Frecuentes encontrados:", len(new_frequent))

    current_frequent = new_frequent
    frequent_itemsets.extend(new_frequent)

fin = time.time()
print(f"\nTiempo de ejecución: {fin - inicio:.2f} segundos")

print("\nTotal itemsets frecuentes encontrados:", len(frequent_itemsets))

```

Total transacciones: 133887
 Soporte mínimo en cantidad: 13388.7

Generando F1...
 Itemsets frecuentes tamaño 1: 16

Generando candidatos tamaño 2...
 Candidatos generados: 120
 Frecuentes encontrados: 36

Generando candidatos tamaño 3...
 Candidatos generados: 83
 Frecuentes encontrados: 59

Generando candidatos tamaño 4...
 Candidatos generados: 73
 Frecuentes encontrados: 70

Generando candidatos tamaño 5...
 Candidatos generados: 56
 Frecuentes encontrados: 56

Generando candidatos tamaño 6...
 Candidatos generados: 28
 Frecuentes encontrados: 28

```
Generando candidatos tamaño 7...
Candidatos generados: 8
Frecuentes encontrados: 8

Generando candidatos tamaño 8...
Candidatos generados: 1
Frecuentes encontrados: 1

Generando candidatos tamaño 9...
Candidatos generados: 0
Frecuentes encontrados: 0

Tiempo de ejecución: 4.03 segundos

Total itemsets frecuentes encontrados: 274
```

2.2.3 Generación Manual de Reglas de Asociación

En esta sección se implementa manualmente el Algoritmo 2, el cual toma los itemsets frecuentes generados anteriormente y construye reglas de asociación que cumplan con un umbral mínimo de confianza.

```
# =====
# Parámetro de confianza
# =====

min_confidence = 0.7

print("\nConfianza mínima:", min_confidence)

# =====
# Crear diccionario de soportes
# =====

# Para poder calcular confidence necesitamos conocer support(I) y support(s)

support_dict = {}

inicio = time.time()
```

```

for itemset in frequent_itemsets:
    support_dict[itemset] = calculate_support(itemset) / n_transactions

# =====
# Generación de reglas
# =====

rules = []

for itemset in frequent_itemsets:

    # Solo conjuntos tamaño >= 2 pueden generar reglas
    if len(itemset) >= 2:

        # Generar todos los subconjuntos no vacíos propios
        for i in range(1, len(itemset)):
            for antecedent in combinations(itemset, i):

                antecedent = frozenset(antecedent)
                consequent = itemset - antecedent

                support_I = support_dict[itemset]
                support_A = support_dict[antecedent]
                support_B = support_dict[consequent]

                confidence = support_I / support_A
                lift = confidence / support_B

                if confidence >= min_confidence:
                    rules.append({
                        "antecedent": antecedent,
                        "consequent": consequent,
                        "support_antecedent": support_A,
                        "support_consequent": support_B,
                        "support_rule": support_I,
                        "confidence": confidence,
                        "lift": lift
                    })

fin = time.time()
print(f"\nTiempo de ejecución: {fin - inicio:.2f} segundos")

```

```

print("\nTotal de reglas encontradas:", len(rules))

# Convertir a DataFrame para visualizar
rules_df = pd.DataFrame(rules)

# Convertir conjuntos a texto para mostrar bonito
rules_df["antecedent"] = rules_df["antecedent"].apply(lambda x: ", ".join(x))
rules_df["consequent"] = rules_df["consequent"].apply(lambda x: ", ".join(x))

rules_df.sort_values(by="confidence", ascending=False).reset_index(drop=True)

```

Confianza mínima: 0.7

Tiempo de ejecución: 3.28 segundos

Total de reglas encontradas: 5938

	antecedent	consequent
0	ESTATUS_VICTIMA_DESCONOCIDO	SEXO_DESCONOCIDO
1	MES_DESAPARICION_DESCONOCIDO, SEXO_DESCONOCIDO...	AÑO_REGISTRO_DESCO...
2	AÑO_REGISTRO_DESCONOCIDO, SEXO_DESCONOCIDO, GR...	MES_DESAPARICION_DI...
3	AÑO_REGISTRO_DESCONOCIDO, MES_DESAPARICION_DES...	SEXO_DESCONOCIDO
4	AÑO_REGISTRO_DESCONOCIDO, MES_DESAPARICION_DES...	GRUPO_EDAD_DESCON...
...	...	...
5933	GRUPO_EDAD_18-29	SEXO_HOMBRE
5934	GRUPO_EDAD_18-29, ESTATUS_VICTIMA_DESAPARECIDA	SEXO_HOMBRE
5935	ESTATUS_VICTIMA_DESAPARECIDA	SEXO_HOMBRE
5936	GRUPO_EDAD_18-29	ESTATUS_VICTIMA_DES...
5937	ESTATUS_VICTIMA_DESAPARECIDA, GRUPO_EDAD_DESCO...	SEXO_HOMBRE

En la siguiente gráfica podemos ver la comparativa entre confianza y lift de cada regla encontrada. Además, es posible comparar qué tan común es el patrón en el conjunto de datos (su soporte), mediante la diferencia del tamaño del punto que lo representa.

```

import plotly.express as px

def graf_reglas(reglas):
    reglas["regla"] = (

```

```

    reglas["antecedent"].str.replace(',', ' <br>') + "<br> → <br>" + reglas["consequent"]
)

fig = px.scatter(
    reglas,
    x="confidence",
    y="lift",
    size="support_rule",
    hover_data={
        "regla": True,
        "support_rule": ":%.3f",
        "confidence": ":%.3f",
        "lift": ":%.3f",
    },
    color="lift",
    color_continuous_scale="RdYlGn",
    labels={
        "confidence": "Confianza",
        "lift": "Lift",
        "support_rule": "Soporte",
    },
    size_max=28,
)

fig.add_hline(
    y=1.0,
    line_dash="dash",
    line_color="crimson",
    annotation_text="Lift = 1 (independencia estadística)",
    annotation_position="top left",
    annotation_font_color="crimson",
)

fig.update_layout(
    height=500,
    xaxis=dict(range=[0, 1.05], title="Confianza"),
    yaxis=dict(title="Lift"),
    coloraxis_colorbar=dict(title="Lift"),
)
return fig

graf_reglas_1 = graf_reglas(rules_df)

```

```
graf_reglas_1.show()
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Como vemos todas nuestras reglas se alejan en al menos 0.4 puntos de la independencia estadística lo que nos indica que la co-ocurrencia del antecedente y el consecuente es en buena medida mayor al azar. Otra observación sencilla es que podemos distinguir tres agrupaciones de las reglas: las verdes de la esquina superior derecha, las naranjas de la esquina inferior izquierda y las naranjas en la esquina inferior derecha.

En primera instancia, lo que quisiéramos hacer es revisar las reglas verdes pues no solo son las de mayor tamaño (frecuencia de aparición) sino también las de mayor confianza y lift lo que podría indicar que serán muy informativas. Sin embargo, si las revisamos a detalle, es posible notar que en realidad están fuertemente relacionadas con que en los registros del conjunto de datos original, cuando se tiene un caso en estado confidencial, se desconoce la fecha de nacimiento, el sexo, la fecha de desaparición, la fecha de registro, el estatus de la víctima y el municipio. Lo que a su vez propicia que se desconozca su pertenencia a las categorías creadas durante la discretización de los datos.

Debido a estas relaciones bien conocidas entre los datos desconocidos, decidimos eliminar las reglas que reflejan las siguientes condiciones:

- Se desconoce el mes de registro y por lo tanto también se desconoce el año (o viceversa)
- Se desconoce el mes de desaparición y por lo tanto también se desconoce el año (o viceversa)
- Se desconoce el grupo de edad y por lo tanto también se desconoce el mes o año de desaparición (o viceversa)

```
# Reglas que desconocen algún campo de la fecha de registro y en consecuencia los otros también
mask_registro = (
    rules_df['antecedent'].str.contains('REGISTRO_DESCONOCIDO', na=False) &
    rules_df['consequent'].str.contains('REGISTRO_DESCONOCIDO', na=False)
)

# Reglas que desconocen algún campo de la fecha de desaparición y en consecuencia los otros también
mask_desap = (
    rules_df['antecedent'].str.contains('DESAPARICION_DESCONOCIDO', na=False) &
    rules_df['consequent'].str.contains('DESAPARICION_DESCONOCIDO', na=False)
)
```

```

# Reglas que desconocen el grupo de edad y en consecuencia la fecha de desaparición
mask_edad_desap = (
    rules_df['antecedent'].str.contains('EDAD_DESCONOCIDO', na=False) &
    rules_df['consequent'].str.contains('DESAPARICION_DESCONOCIDO', na=False)
)

# Reglas que desconocen la fecha de desaparición y en consecuencia el grupo de edad
mask_desap_edad = (
    rules_df['antecedent'].str.contains('DESAPARICION_DESCONOCIDO', na=False) &
    rules_df['consequent'].str.contains('EDAD_DESCONOCIDO', na=False)
)

mask_total = mask_registro | mask_desap | mask_edad_desap | mask_desap_edad

# Filtrar
rules_filtradas = rules_df[~mask_total]

rules_filtradas

```

	antecedent	consequent
0	SEXO_MUJER	ESTATUS_VICTIMA_DES...
1	GRUPO_EDAD_30-44	SEXO_HOMBRE
2	MUNICIPIO_DESCONOCIDO	ESTATUS_VICTIMA_DES...
3	ESTATUS_VICTIMA_DESCONOCIDO	MUNICIPIO_DESCONOCI...
4	AÑO_DESAPARICION_DESCONOCIDO	ESTATUS_VICTIMA_DES...
...	...	...
5915	AÑO_REGISTRO_DESCONOCIDO, MES_DESAPARICION_DES...	SEXO_DESCONOCIDO, M...
5925	MES_DESAPARICION_DESCONOCIDO, SEXO_DESCONOCIDO...	AÑO_REGISTRO_DESCO...
5930	AÑO_REGISTRO_DESCONOCIDO, MES_DESAPARICION_DES...	ESTATUS_VICTIMA_DES...
5933	AÑO_REGISTRO_DESCONOCIDO, MES_DESAPARICION_DES...	MUNICIPIO_DESCONOCI...
5935	AÑO_REGISTRO_DESCONOCIDO, MES_DESAPARICION_DES...	SEXO_DESCONOCIDO

Aunque este proceso de filtrado logró reducir el número de reglas de 5,938 a 2,308. Aún es posible encontrar asociaciones en las que se ve el efecto de que tener datos confidenciales nos genera desconocimiento en otros atributos. Por ello, decidimos descartar todas las reglas cuyo antecedente es un dato desconocido y cuyo consecuente es otro dato desconocido.

```

mask_total = (
    rules_filtradas['antecedent'].str.contains('DESCONOCIDO', na=False) &
    rules_filtradas['consequent'].str.contains('DESCONOCIDO', na=False)
)

```

```
)
rules_filtradas = rules_filtradas[~mask_total]

rules_filtradas.sort_values(
    by=["support_rule", "confidence", "lift"],
    ascending=False
)
```

	antecedent	consequent
8	SEXO_HOMBRE	ESTATUS_VICTIMA_DESA
7	ESTATUS_VICTIMA_DESAPARECIDA	SEXO_HOMBRE
279	SEXO_HOMBRE, GRUPO_EDAD_DESCONOCIDO	ESTATUS_VICTIMA_DESA
278	ESTATUS_VICTIMA_DESAPARECIDA, GRUPO_EDAD_DESCO...	SEXO_HOMBRE
0	SEXO_MUJER	ESTATUS_VICTIMA_DESA
32	GRUPO_EDAD_18-29	ESTATUS_VICTIMA_DESA
31	GRUPO_EDAD_30-44	ESTATUS_VICTIMA_DESA
33	GRUPO_EDAD_18-29	SEXO_HOMBRE
1	GRUPO_EDAD_30-44	SEXO_HOMBRE
119	GRUPO_EDAD_18-29, SEXO_HOMBRE	ESTATUS_VICTIMA_DESA
118	GRUPO_EDAD_18-29, ESTATUS_VICTIMA_DESAPARECIDA	SEXO_HOMBRE
117	GRUPO_EDAD_18-29	ESTATUS_VICTIMA_DESA
266	GRUPO_EDAD_30-44, SEXO_HOMBRE	ESTATUS_VICTIMA_DESA
265	ESTATUS_VICTIMA_DESAPARECIDA, GRUPO_EDAD_30-44	SEXO_HOMBRE
264	GRUPO_EDAD_30-44	ESTATUS_VICTIMA_DESA

Después de este último filtrado obtenemos no solo un menor número de reglas sino asociaciones más interesantes pues ya no reflejan patrones relacionados a la forma en la que se registran los datos sino en el comportamiento real de los mismos.

Ahora bien, si comparamos las métricas de cada regla podemos ver que todas cumplen con los umbrales definidos (soporte mínimo de 0.1 y confianza de 0.7) gracias al algoritmo aplicado. También en todas se cumple que la **confianza es mayor a la probabilidad del consecuente** por lo que podemos asegurar que el antecedente tiene influencia en el consecuente.

```
graf_reglas_3 = graf_reglas(rules_filtradas)
graf_reglas_3.show()
```

Unable to display output for mime type(s): text/html

Las reglas más comunes en el dataset son:

- **SEXO_HOMBRE $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA** y **ESTATUS_VICTIMA_DESAPARECIDA $\Rightarrow$ SEXO_HOMBRE**
- **SEXO_HOMBRE, GRUPO_EDAD_DESCONOCIDO $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA** y **ESTATUS_VICTIMA_DESAPARECIDA, GRUPO_EDAD_DESCONOCIDO $\Rightarrow$ SEXO_HOMBRE**

Si bien en ambos grupos el soporte de ambas reglas es el mismo, la primera regla de cada grupo tiene una confianza mayor que la segunda, por lo que podemos interpretarlas como:

- **Si una víctima es hombre es 94% probable que su estatus sea desaparecido**, mientras que si una víctima tiene estatus desaparecida es un 76% probable que sea hombre
- **Si una víctima es hombre y se desconoce su grupo de edad es 93% probable que su estatus sea desaparecida**, mientras que si sabemos que está desaparecida y que se desconoce su edad es 71% probable que sea hombre.

Esto lo que nos indica es que en efecto **los registros de hombres tienen una muy alta probabilidad de pertenecer al estatus desaparecido** independientemente de su grupo de edad (pues solo hay un cambio del 1%), **pero** ese cambio de aproximadamente el 20% al “voltear” las reglas, nos muestra que **el estatus desaparecida no es exclusivo de los hombres**.

Otras reglas que tenemos son:

- **SEXO_MUJER $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA**: Si la víctima **es mujer es 94% probable que su estatus sea desaparecida** y es un patrón presente en casi el 14% de nuestros datos
- **GRUPO_EDAD_18-29 $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA**: Si la víctima tiene **entre 18 y 29 años es 94% probable que su estatus sea desaparecida** y es un patrón presente en casi el 13% de nuestros datos
- **GRUPO_EDAD_30-44 $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA**: Si la víctima tiene **entre 30 y 44 años es 94% probable que su estatus sea desaparecida** y es un patrón presente en el 12% de nuestros datos
- **GRUPO_EDAD_18-29 $\Rightarrow$ SEXO_HOMBRE**: Si la víctima tiene entre **18 y 29 años es 80% probable que sea hombre** y es un patrón presente en el 11% de nuestros datos
- **GRUPO_EDAD_30-44 $\Rightarrow$ SEXO_HOMBRE**: Si la víctima tiene entre **30 y 44 años es 86% probable que sea hombre** y es un patrón presente en el 11% de nuestros datos
- **GRUPO_EDAD_18-29, SEXO_HOMBRE $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA**: Si la víctima tiene entre **18 y 29 años y es hombre, es 94% probable que su estatus sea desaparecida** y es un patrón presente en el 10% de nuestros datos

- GRUPO_EDAD_18-29, ESTATUS_VICTIMA_DESAPARECIDA $\Rightarrow$ SEXO_HOMBRE: Si la víctima tiene entre **18 y 29 años y su estatus es desaparecida es 80% probable que sea hombre** y es un patrón presente en el 10% de nuestros datos
- GRUPO_EDAD_18-29 $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA, SEXO_HOMBRE: Si la víctima tiene entre **18 y 29 años es 76% probable que su estatus sea desaparecida y sea hombre** y es un patrón presente en el 10% de nuestros datos
- GRUPO_EDAD_30-44, SEXO_HOMBRE $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA: Si la víctima tiene entre **30 y 44 años y es hombre es 94% probable que su estatus sea desaparecida** y es un patrón presente en el 10% de nuestros datos
- GRUPO_EDAD_30-44, ESTATUS_VICTIMA_DESAPARECIDA $\Rightarrow$ SEXO_HOMBRE: Si la víctima tiene **estatus desaparecida y entre 30 y 44 años es 86% probable que sea hombre** y es un patrón presente en el 10% de nuestros datos
- GRUPO_EDAD_30-44 $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA, SEXO_HOMBRE: Si la víctima tiene entre **30 y 44 años es 94% probable que su estatus sea desaparecida y que sea hombre** y es un patrón presente en el 10% de nuestros datos

De estas podemos ver que se refleja el desbalance que existe en el conjunto de datos original pues sabemos que estatus de la víctima más común es el de desaparecida.

```
df_original = pd.read_csv("../data/data_secretariado.csv")

# frecuencias absolutas
conteo = df_original['ESTATUS_VICTIMA'].value_counts().reset_index()
conteo.columns = ['Estatus', 'Número de registros']

# media y mediana
s = conteo['Número de registros'].dropna()
media = s.mean()
mediana = s.median()

# Gráfica
fig = px.bar(
    conteo,
    x = 'Estatus',
    y = 'Número de registros',
    title = 'Número de registros por estatus'
)

# Línea de la media
fig.add_hline(
    y=media,
```

```

        line_dash="dash",
        line_width=2,
        annotation_text=f"Media = {media:.2f}",
        annotation_position="bottom right"
    )

# Línea de la mediana
fig.add_hline(
    y=mediana,
    line_dash="dot",
    line_width=2,
    annotation_text=f"Mediana = {mediana:.2f}",
    annotation_position="top right"
)

fig.update_layout(
    xaxis_title = 'Estatus',
    yaxis_title = 'Número de registros',
    xaxis={'categoryorder': 'total descending'}
)

fig.show()

```

Unable to display output for mime type(s): text/html

2.3 Conclusión del Algoritmo Apriori Manual

A partir de la implementación manual del algoritmo Apriori se trabajó con un total de **133,887 transacciones** y un soporte mínimo equivalente al **10%** (13,388 registros aproximadamente).

Durante la generación de itemsets frecuentes, el algoritmo avanzó nivel por nivel hasta que ya no fue posible generar nuevos candidatos. El proceso produjo:

- 16 itemsets frecuentes de tamaño 1.
- 36 itemsets frecuentes de tamaño 2.
- 59 itemsets frecuentes de tamaño 3.
- 70 itemsets frecuentes de tamaño 4.
- 56 itemsets frecuentes de tamaño 5.

- 28 itemsets frecuentes de tamaño 6.
- 8 itemsets frecuentes de tamaño 7.
- 1 itemset frecuente de tamaño 8.

En total se obtuvieron **274 itemsets frecuentes**.

Esto muestra cómo el algoritmo va reduciendo progresivamente el número de combinaciones conforme aumenta el tamaño del conjunto, hasta que ya no se generan nuevos candidatos.

Posteriormente, utilizando un umbral de confianza mínima del **70%**, se aplicó el segundo procedimiento del algoritmo para generar reglas de asociación fuertes. Como resultado, se obtuvieron **5,938 reglas fuertes**.

Las reglas se interpretan principalmente con tres métricas:

- **Support (Soporte):** Indica qué tan frecuente es una combinación dentro del total de registros.
- **Confidence (Confianza):** Mide qué tan probable es que ocurra el consecuente cuando ocurre el antecedente.
- **Lift:** Indica qué tan fuerte es la relación entre antecedente y consecuente.
 - $\text{Lift} > 1 \rightarrow$ Asociación positiva.
 - $\text{Lift} = 1 \rightarrow$ Independencia.
 - $\text{Lift} < 1 \rightarrow$ Asociación negativa.

En los resultados se observa que muchas reglas con confianza igual a 1.0 están relacionadas con categorías marcadas como “DESCONOCIDO”. Esto indica que existe una alta concentración de valores faltantes o no especificados dentro del conjunto de datos, lo cual influye directamente en los patrones detectados.

También se identificaron reglas más interpretables, por ejemplo:

- El estatus “DESAPARECIDA” muestra una fuerte relación con el sexo masculino aunque no es exclusivo de este sexo pues una víctima mujer también tiene un 94% de probabilidad de tener dicho estatus.
- Personas de los grupos de edad 18–29 y 30–44 presentan una alta probabilidad, 80% y 86% respectivamente, de estar asociadas al sexo masculino y, por ende, al estatus “DESAPARECIDA”.

En general, los resultados confirman que el algoritmo logra identificar combinaciones frecuentes y relaciones condicionales relevantes dentro del dataset, aunque la calidad de los datos (especialmente los valores “DESCONOCIDO”) influye considerablemente en los patrones obtenidos.

La implementación manual permitió comprender con mayor profundidad la lógica interna del algoritmo, especialmente el proceso de generación iterativa y el cálculo explícito de soporte y confianza.

- **Generación de itemsets frecuentes** ≈ 4.29 segundos
- **Generación de reglas** ≈ 3.40 segundos

3 Aplicación del algoritmo Apriori de python

En esta sección se aplica el algoritmo **Apriori** implementado en `mlxtend.frequent_patterns` con el objetivo de identificar patrones frecuentes y generar reglas de asociación dentro del conjunto de datos previamente transformado a valores booleanos.

Cada fila del dataset representa una transacción (registro individual) y cada columna representa un ítem (sexo, grupo de edad, entidad, municipio, año, etc.).

Dado que el dataset contiene 133,887 registros y miles de columnas, es importante definir adecuadamente los parámetros del algoritmo para evitar ruido y sobreajuste.

3.1 Carga de datos

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# Cargar dataset
df = pd.read_csv("../data/data_secretariado_apriori.csv")

print("Dimensiones del dataset:", df.shape)
```

Dimensiones del dataset: (133887, 1825)

3.2 Definición de parámetros

Si bien en la sección anterior se obtuvieron algunas reglas importantes, para esta segunda aplicación del algoritmo lo que hicimos fue relajar un poco el soporte con el fin de permitir una menor ocurrencia de los patrones pero manteniendo la confianza, como un intento de encontrar patrones un poco menos comunes pero con el mismo nivel de influencia entre antecedentes y consecuentes.

Soporte mínimo

Se define un soporte mínimo de **5% (0.05)**.

Justificación:

- El dataset contiene 133,887 registros, un soporte de 5% implica que un patrón debe aparecer en al menos 66,944 casos para considerarse relevante.
- Inicialmente se probó con un soporte de 2% (0.02), lo que generó 4,980 itemsets frecuentes y 134,854 reglas de asociación, dificultando el análisis e interpretación de resultados.
- Al aumentar el soporte a 5%, se redujo considerablemente el número de resultados, permitiendo enfocarse en buscar patrones más sólidos y representativos del comportamiento general de los datos.

Confianza mínima

Se define una confianza mínima de **70% (0.7)**.

Justificación:

- Una confianza de 70% garantiza que, dado el antecedente, el consecuente ocurre con alta probabilidad.
- Permite descartar reglas débiles o poco consistentes.
- Aumenta la interpretabilidad de resultados.

Tamaño máximo de los itemsets

Se limitó el tamaño máximo de los itemsets a tres elementos (`max_len = 3`).

Justificación:

- Como vimos el algoritmo Apriori genera combinaciones de manera progresiva, y mientras más elementos tenga cada conjunto, el número de posibles combinaciones crece muy rápido.
- Si no se pone un límite, el algoritmo puede empezar a generar reglas demasiado complejas o difíciles de interpretar.
- Facilita la interpretación de los patrones encontrados.
- Evita generar reglas excesivamente específicas o difíciles de analizar, por ejemplo: Sabemos que un dato confidencial implica hasta 5 datos confidenciales más, el limitar a un número menor de 6 items elimina desde el inicio la posibilidad de generar reglas con ese comportamiento

3.3 Generación de itemsets frecuentes

```
import os
import pandas as pd
import time

from mlxtend.frequent_patterns import apriori, association_rules

pd.set_option('display.max_columns', None)
pd.set_option('display.max_colwidth', None)
pd.set_option('display.width', None)

min_support = 0.05

inicio = time.time()
frequent_itemsets = apriori(
    df,
    min_support=min_support,
    use_colnames=True,
    max_len=3
)

fin = time.time()
print(f"Tiempo de ejecución: {fin - inicio:.2f} segundos")

print("Total itemsets encontrados:", len(frequent_itemsets))

# Ordenar por soporte y resetear índice
frequent_itemsets = frequent_itemsets.sort_values(
    by="support",
    ascending=False
).reset_index(drop=True)

# Guardar versión original
frequent_itemsets.to_csv("../data/itemsets_frecuentes.csv", index=False)

# Crear copia SOLO para mostrar bonito
itemsets_display = frequent_itemsets.copy()
itemsets_display['itemsets'] = itemsets_display['itemsets'].apply(
    lambda x: ', '.join(list(x))
)
```



```
itemsets_display
```

Tiempo de ejecución: 0.42 segundos

Total itemsets encontrados: 211

	support	itemsets
0	0.619844	GRUPO_EDAD_DESCONOCIDO
1	0.596630	ESTATUS_VICTIMA_DESAPARECIDA
2	0.482989	SEXO_HOMBRE
3	0.454189	SEXO_HOMBRE, ESTATUS_VICTIMA_DESAPARECIDA
4	0.428040	AÑO_DESAPARICION_DESCONOCIDO
...	...	...
206	0.050341	AÑO_REGISTRO_DESCONOCIDO, ORIGEN_REPORTE_PROCURADURIA_GENERAL
207	0.050341	MES_REGISTRO_DESCONOCIDO, ORIGEN_REPORTE_PROCURADURIA_GENERAL
208	0.050341	AÑO_REGISTRO_DESCONOCIDO, MES_REGISTRO_DESCONOCIDO, ORIGEN_REPORTE_PROCURADURIA_GENERAL
209	0.050102	ENTIDAD_TAMAULIPAS, MES_REGISTRO_DESCONOCIDO, ORIGEN_REPORTE_PROCURADURIA_GENERAL
210	0.050102	AÑO_REGISTRO_DESCONOCIDO, ENTIDAD_TAMAULIPAS, ORIGEN_REPORTE_PROCURADURIA_GENERAL

3.4 Generación de Reglas de Asociación

```
min_confidence = 0.7

inicio = time.time()
rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=min_confidence
)

fin = time.time()
print(f"Tiempo de ejecución: {fin - inicio:.2f} segundos")

print("Total reglas encontradas:", len(rules))

# Seleccionar solo métricas importantes
```

```

rules = rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]

# Convertir a texto legible
rules['antecedents'] = rules['antecedents'].apply(
    lambda x: ', '.join(list(x))
)
rules['consequents'] = rules['consequents'].apply(
    lambda x: ', '.join(list(x))
)

# Ordenar por lift y resetear índice
rules = rules.sort_values(
    by="lift",
    ascending=False
).reset_index(drop=True)

# Guardar versión final
rules.to_csv("../data/reglas_fuertes.csv", index=False)

rules

```

Tiempo de ejecución: 0.00 segundos

Total reglas encontradas: 490

	antecedents
0	AÑO_DESAPARICION_2025.0
1	AÑO_REGISTRO_2025.0
2	AÑO_DESAPARICION_2025.0, ESTATUS_VICTIMA_DESAPARECIDA
3	AÑO_REGISTRO_2025.0
4	AÑO_DESAPARICION_2025.0
...	...
485	ORIGEN_REPORTE_FISCALIA GENERAL DE JALISCO
486	ENTIDAD_ESTADO DE MÉXICO
487	ORIGEN_REPORTE_PROCURADURIA GENERAL DE JUSTICIA DEL ESTADO DE TAMAULIPAS
488	ENTIDAD_TAMAULIPAS, ORIGEN_REPORTE_PROCURADURIA GENERAL DE JUSTICIA DE
489	ENTIDAD_TAMAULIPAS

```

import plotly.express as px

def graf_reglas(reglas):

```

```

reglas["regla"] = (
    reglas["antecedents"].str.replace(',', ' <br>') + "<br> → <br>" + reglas["consequents"]
)

fig = px.scatter(
    reglas,
    x="confidence",
    y="lift",
    size="support",
    hover_data={
        "regla": True,
        "support": ":.3f",
        "confidence": ":.3f",
        "lift": ":.3f",
    },
    color="lift",
    color_continuous_scale="RdYlGn",
    labels={
        "confidence": "Confianza",
        "lift": "Lift",
        "support": "Soporte",
    },
    size_max=28,
)

fig.add_hline(
    y=1.0,
    line_dash="dash",
    line_color="crimson",
    annotation_text="Lift = 1 (independencia estadística)",
    annotation_position="top left",
    annotation_font_color="crimson",
)

fig.update_layout(
    height=500,
    xaxis=dict(range=[0, 1.05], title="Confianza"),
    yaxis=dict(title="Lift"),
    coloraxis_colorbar=dict(title="Lift"),
)

return fig

```

```
graf_reglas_1 = graf_reglas(rules)
graf_reglas_1.show()
```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Nuevamente tenemos muchas reglas que reflejan los patrones ya conocidos de los datos faltantes, por lo que filtramos quitando aquellas que reflejan que un dato desconocido propicia otro dato desconocido.

```
mask_desconocidos = (
    rules['antecedents'].str.contains('DESCONOCIDO', na=False) &
    rules['consequents'].str.contains('DESCONOCIDO', na=False)
)
rules_filtradas = rules[~mask_desconocidos]

rules_filtradas.sort_values(
    by=["support", "confidence", "lift"],
    ascending=False
)

graf_reglas_1 = graf_reglas(rules_filtradas)
graf_reglas_1.show()
```

Unable to display output for mime type(s): text/html

De las reglas restantes muchas de ellas asocian la entidad y el origen del reporte pues comúnmente son del mismo estado, sin embargo este es un comportamiento un poco obvio pues en la mayoría de los casos el lugar de desaparición será el lugar donde se hará el reporte. Por ello, decidimos eliminar las reglas que contenían el mismo estado en ambos lados. Tomando en cuenta que los estados más comunes son Jalisco, Estado de México y Tamaulipas.

```
df_original = pd.read_csv("../data/data_secretariado.csv")

# frecuencias absolutas
conteo = df_original['ENTIDAD'].value_counts().reset_index()
conteo.columns = ['Entidad', 'Número de registros']

# media y mediana
```

```

s = conteo['Número de registros'].dropna()
media = s.mean()
mediana = s.median()

# Gráfica
fig = px.bar(
    conteo,
    x = 'Entidad',
    y = 'Número de registros',
    title = 'Número de registros por entidad'
)

# Línea de la media
fig.add_hline(
    y=media,
    line_dash="dash",
    line_width=2,
    annotation_text=f"Media = {media:.2f}",
    annotation_position="bottom right"
)

# Línea de la mediana
fig.add_hline(
    y=mediana,
    line_dash="dot",
    line_width=2,
    annotation_text=f"Mediana = {mediana:.2f}",
    annotation_position="top right"
)

fig.update_layout(
    xaxis_title = 'Entidad',
    yaxis_title = 'Número de registros',
    xaxis={'categoryorder': 'total descending'}
)

fig.show()

```

Unable to display output for mime type(s): text/html

```

mask_estados = (
    (rules_filtradas['antecedents'].str.contains('JALISCO', na=False) &
    rules_filtradas['consequents'].str.contains('JALISCO', na=False)) |
    (rules_filtradas['antecedents'].str.contains('ESTADO DE MÉXICO', na=False) &
    rules_filtradas['consequents'].str.contains('ESTADO DE MÉXICO', na=False)) |
    (rules_filtradas['antecedents'].str.contains('TAMAULIPAS', na=False) &
    rules_filtradas['consequents'].str.contains('TAMAULIPAS', na=False))
)

rules_filtradas2 = rules_filtradas[~mask_estados]

rules_filtradas2.sort_values(
    by=["support", "confidence", "lift"],
    ascending=False
)

graf_reglas_1 = graf_reglas(rules_filtradas2)
graf_reglas_1.show()

```

Unable to display output for mime type(s): text/html

Dado que tenemos algunas que se asocian porque el año de registro y desaparición es el mismo, también decidimos descartarlas.

```

import re

def mismo_anio(row):
    ant = row['antecedents']
    cons = row['consequents']

    # Extraer año (4 dígitos)
    ant_year = re.search(r'\d{4}', ant)
    cons_year = re.search(r'\d{4}', cons)

    if ant_year and cons_year:
        return ant_year.group() == cons_year.group()

    return False

mask_mismo_anio = rules_filtradas2.apply(mismo_anio, axis=1)
rules_filtradas2 = rules_filtradas2[~mask_mismo_anio]

```

En este punto podemos ver que existe un **problema de consistencia** al momento de registrar a la “COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE MEXICO” pues hace falta el acento y por ello no fue filtrada correctamente la regla que la asocia a “ENTIDAD_ESTADO DE MÉXICO”.

```
rules_filtradas2 = rules_filtradas2.drop(rules_filtradas2.index[0])

rules_filtradas2 = rules_filtradas2.sort_values(
    by=["support", "confidence", "lift"],
    ascending=False
)
```

Las primeras 15 reglas obtenidas son las que ya analizamos en la sección anterior, por lo que nos centramos en las restantes:

```
rules_filtradas2 = rules_filtradas2.drop(rules_filtradas2.index[:15])
rules_filtradas2
```

	antecedents
484	ENTIDAD_JALISCO
486	ENTIDAD_ESTADO DE MÉXICO
489	ENTIDAD_TAMAULIPAS
487	ORIGEN_REPORTE_PROCURADURIA GENERAL DE JUSTICIA DEL ESTADO DE TAMAULIPAS
488	ENTIDAD_TAMAULIPAS, ORIGEN_REPORTE_PROCURADURIA GENERAL DE JUSTICIA DE
477	GRUPO_EDAD_DESCONOCIDO, SEXO_MUJER
485	ORIGEN_REPORTE_FISCALIA GENERAL DE JALISCO
483	ENTIDAD_JALISCO, ORIGEN_REPORTE_FISCALIA GENERAL DE JALISCO
428	AÑO_REGISTRO_2024.0
429	AÑO_DESAPARICION_2024.0
430	AÑO_DESAPARICION_2024.0, AÑO_REGISTRO_2024.0
424	AÑO_REGISTRO_2025.0
425	AÑO_DESAPARICION_2025.0
426	AÑO_REGISTRO_2025.0, AÑO_DESAPARICION_2025.0
466	MES_REGISTRO_Mayo
470	MES_REGISTRO_Junio

- ENTIDAD_JALISCO $\Rightarrow$ GRUPO_EDAD_DESCONOCIDO: Si la víctima pertenece a **Jalisco es 83% probable que su grupo de edad sea desconocido** y es un patrón presente en aproximadamente el 9% de los datos
- ENTIDAD_ESTADO DE MÉXICO $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA: Si la víctima pertenece al **Estado de México es 72% probable que su estatus sea desaparecida** y es un patrón presente en casi el 7.7% de los datos

- ENTIDAD_TAMAULIPAS $\Rightarrow$ GRUPO_EDAD_DESCONOCIDO: Si la víctima pertenece a **Tamaulipas es 72% probable que su grupo de edad sea desconocido** y es un patrón presente en aproximadamente el 7.2% de los datos
- ORIGEN_REPORTE_PROCURADURIA_GENERAL_DE_JUSTICIA_DEL_ESTADO_DE_TAMAULIPAS $\Rightarrow$ GRUPO_EDAD_DESCONOCIDO: Si el registro proviene de la **Procuraduría General de Justicia del Estado de Tamaulipas es 73% probable que el grupo de edad sea desconocido** y es un patrón presente en el 6.6% de los datos
- GRUPO_EDAD_DESCONOCIDO, SEXO_MUJER $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA: Si la víctima es **mujer y su grupo de edad es desconocido es 94% probable que su estatus sea desaparecida** y es un patrón presente en el 6.4% de los datos
- ORIGEN_REPORTE_FISCALIA_GENERAL_DE_JALISCO $\Rightarrow$ GRUPO_EDAD_DESCONOCIDO: Si el registro proviene de la **Fiscalía General de Jalisco es 83% probable que el grupo de edad sea desconocido** y es un patrón presente en el 6.4% de los datos
- AÑO_REGISTRO_2024 $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA: Si el **registro corresponde al año 2024 es 99.6% probable que el estatus sea desaparecida** y es un patrón presente en el 6.0% de los datos
- AÑO_DESAPARICION_2024 $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA: Si la **desaparición ocurrió en 2024 es 99.6% probable que el estatus sea desaparecida** y es un patrón presente en el 5.9% de los datos
- AÑO_REGISTRO_2025 $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA: Si el **registro corresponde a 2025 es 99.8% probable que el estatus sea desaparecida** y es un patrón presente en el 5.1% de los datos
- AÑO_DESAPARICION_2025 $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA: Si la **desaparición ocurrió en 2025 es 99.8% probable que el estatus sea desaparecida** y es un patrón presente en el 5.1% de los datos
- MES_REGISTRO_Mayo $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA: Si el **registro ocurrió en mayo es 94.7% probable que el estatus sea desaparecida** y es un patrón presente en el 5.0% de los datos
- MES_REGISTRO_Junio $\Rightarrow$ ESTATUS_VICTIMA_DESAPARECIDA: Si el **registro ocurrió en junio es 94.0% probable que el estatus sea desaparecida** y es un patrón presente en el 5.0% de los datos

Aunque estas reglas son definitivamente más específicas que las de la sección pasada, si las vemos en conjunto podemos notar que una vez más están reflejando la alta presencia del estatus de la víctima como desaparecida y la ausencia de las fechas para el cálculo de la edad.

3.5 Conclusión del Algoritmo Apriori de Python

Con los parámetros definidos (soporte mínimo de 5% y confianza mínima de 70%) se obtuvieron 211 itemsets frecuentes y 490 reglas de asociación, lo cual permitió comparar dos valores para el soporte mínimo. Para el análisis de las reglas, nos basamos en el soporte, la confianza y el lift de cada una.

La aplicación del algoritmo en este caso con un parámetro más flexible no permitió identificar patrones significativamente más informativos. No obstante, ajustar el soporte al 5% fue clave para reducir la cantidad de combinaciones generadas, ya que con un soporte de 2% se obtenían miles de itemsets y más de cien mil reglas, lo que hacía difícil llegar a conclusiones claras.

Limitar el tamaño máximo de los itemsets a tres elementos también ayudó a mantener el análisis enfocado y comprensible, evitando combinaciones demasiado específicas.

En general, el proceso mostró la importancia de elegir correctamente los parámetros y considerar la calidad de los datos al interpretar los resultados.

- **Generación de itemsets frecuentes** ≈ 0.73 segundos
- **Generación de reglas** ≈ 0.00 segundos

4 Aplicación del algoritmo FP-Growth

En esta sección se aplica el algoritmo **FP-Growth** con el objetivo de identificar patrones frecuentes y generar reglas de asociación dentro del conjunto de datos previamente transformado a valores booleanos.

Cada fila del dataset representa una transacción (registro individual) y cada columna representa un ítem (sexo, grupo de edad, entidad, municipio, año, etc.).

Dado que el dataset contiene 133,887 registros y miles de columnas, es importante definir adecuadamente los parámetros del algoritmo para evitar ruido y sobreajuste.

4.1 Carga de datos

```
import pandas as pd
import time
from mlxtend.frequent_patterns import fpgrowth
from mlxtend.frequent_patterns import association_rules

# Cargar dataset
df = pd.read_csv("../data/data_secretariado_apriori.csv")

print("Dimensiones del dataset:", df.shape)
```

Dimensiones del dataset: (133887, 1825)

4.2 Definición de parámetros

En este caso se mantuvieron los parámetros de la implementación de Apriori pues bajar el soporte dispararía el número de reglas y subirlo nos orilla a obtener reglas centradas en las categorías más presentes en el dataset original.

4.2.1 Soporte mínimo

Se define un soporte mínimo de **10% (0.1)**.

Justificación:

- El dataset contiene 133,887 registros, un soporte de 10% implica que un patrón debe aparecer en al menos 13,388 casos para considerarse relevante.
- Un porcentaje más alto resulta muy restrictivo debido al gran desbalance que hay en los datos, tanto por los datos desconocidos como por el estatus de víctima desaparecida.

4.2.2 Confianza mínima

Se define una confianza mínima de **70% (0.7)**.

Justificación:

- Una confianza de 70% garantiza que, dado el antecedente, el consecuente ocurre con alta probabilidad.
 - Permite descartar reglas débiles o poco consistentes.
 - Aumenta la interpretabilidad de resultados.
-

4.3 Generación de itemsets frecuentes

```
inicio = time.time()
frequent_itemsets = fpgrowth(
    df,
    min_support=0.1,
    use_colnames=True
)

frequent_itemsets
```

```
fin = time.time()
print(f"Tiempo de ejecución: {fin - inicio:.2f} segundos")
```

Tiempo de ejecución: 1.12 segundos

4.4 Generación de Reglas de Asociación

```
min_confidence = 0.7

inicio = time.time()

rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=min_confidence
)

fin = time.time()
print(f"Tiempo de ejecución: {fin - inicio:.2f} segundos")
print("\nTotal reglas encontradas:", len(rules))

# Seleccionar solo métricas importantes
rules = rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']]

# Convertir a texto legible
rules['antecedents'] = rules['antecedents'].apply(
    lambda x: ', '.join(list(x))
)
rules['consequents'] = rules['consequents'].apply(
    lambda x: ', '.join(list(x))
)

# Ordenar por lift y resetear índice
rules = rules.sort_values(
    by="lift",
    ascending=False
).reset_index(drop=True)
```

rules

Tiempo de ejecución: 0.01 segundos

Total reglas encontradas: 5938

	antecedents	consequents
0	ESTATUS_VICTIMA_DESCONOCIDO, AÑO_REGISTRO_DESC...	MES_DESAPARICION_D
1	MES_DESAPARICION_DESCONOCIDO, MES_REGISTRO_DESC...	AÑO_DESAPARICION_D
2	ESTATUS_VICTIMA_DESCONOCIDO, MES_REGISTRO_DESC...	AÑO_DESAPARICION_D
3	MES_DESAPARICION_DESCONOCIDO, AÑO_DESAPARICION...	GRUPO_EDAD_DESCON
4	ESTATUS_VICTIMA_DESCONOCIDO, MES_REGISTRO_DESC...	AÑO_DESAPARICION_D
...	...	...
5933	ESTATUS_VICTIMA_DESAPARECIDA	SEXO_HOMBRE
5934	SEXO_HOMBRE	ESTATUS_VICTIMA_DES
5935	MUNICIPIO_DESCONOCIDO	GRUPO_EDAD_DESCON
5936	GRUPO_EDAD_DESCONOCIDO, SEXO_HOMBRE	ESTATUS_VICTIMA_DES
5937	GRUPO_EDAD_DESCONOCIDO, ESTATUS_VICTIMA_DESAPA...	SEXO_HOMBRE

```
import plotly.express as px

def graf_reglas(reglas):
    reglas["regla"] = (
        reglas["antecedents"].str.replace(',', ' <br>') + " <br> → <br>" + reglas["consequents"]
    )

    fig = px.scatter(
        reglas,
        x="confidence",
        y="lift",
        size="support",
        hover_data={
            "regla": True,
            "support": ":.3f",
            "confidence": ":.3f",
            "lift": ":.3f",
        },
        color="lift",
        color_continuous_scale="RdYlGn",
```

```

        labels={
            "confidence": "Confianza",
            "lift": "Lift",
            "support": "Soporte",
        },
        size_max=28,
    )

    fig.add_hline(
        y=1.0,
        line_dash="dash",
        line_color="crimson",
        annotation_text="Lift = 1 (independencia estadística)",
        annotation_position="top left",
        annotation_font_color="crimson",
    )

    fig.update_layout(
        height=500,
        xaxis=dict(range=[0, 1.05], title="Confianza"),
        yaxis=dict(title="Lift"),
        coloraxis_colorbar=dict(title="Lift"),
    )
    return fig

graf_reglas_1 = graf_reglas(rules)
graf_reglas_1.show()

```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Al quitar las reglas que relacionan datos desconocidos con otros datos desconocidos, obtenemos las mismas reglas que en la aplicación del algoritmo Apriori que implementamos.

```

mask_total = (
    rules['antecedents'].str.contains('DESCONOCIDO', na=False) &
    rules['consequents'].str.contains('DESCONOCIDO', na=False)
)
rules_filtradas = rules[~mask_total]

```

```

rules_filtradas.sort_values(
    by=["support", "confidence", "lift"],
    ascending=False
)

graf_reglas_1 = graf_reglas(rules_filtradas)
graf_reglas_1.show()

rules_filtradas

```

Unable to display output for mime type(s): text/html

	antecedents	consequents
5796	GRUPO_EDAD_30-44	SEXO_HOMBRE, ESTATUS_VICTIMA_DESAPARECIDA
5797	GRUPO_EDAD_30-44	SEXO_HOMBRE
5798	GRUPO_EDAD_30-44, ESTATUS_VICTIMA_DESAPARECIDA	SEXO_HOMBRE
5799	GRUPO_EDAD_18-29	SEXO_HOMBRE, ESTATUS_VICTIMA_DESAPARECIDA
5800	GRUPO_EDAD_18-29	SEXO_HOMBRE
5801	GRUPO_EDAD_18-29, ESTATUS_VICTIMA_DESAPARECIDA	SEXO_HOMBRE
5925	SEXO_MUJER	ESTATUS_VICTIMA_DESAPARECIDA
5926	GRUPO_EDAD_18-29	ESTATUS_VICTIMA_DESAPARECIDA
5927	GRUPO_EDAD_18-29, SEXO_HOMBRE	ESTATUS_VICTIMA_DESAPARECIDA
5928	GRUPO_EDAD_30-44	ESTATUS_VICTIMA_DESAPARECIDA
5932	SEXO_HOMBRE, GRUPO_EDAD_30-44	ESTATUS_VICTIMA_DESAPARECIDA
5933	ESTATUS_VICTIMA_DESAPARECIDA	SEXO_HOMBRE
5934	SEXO_HOMBRE	ESTATUS_VICTIMA_DESAPARECIDA
5936	GRUPO_EDAD_DESCONOCIDO, SEXO_HOMBRE	ESTATUS_VICTIMA_DESAPARECIDA
5937	GRUPO_EDAD_DESCONOCIDO, ESTATUS_VICTIMA_DESAPARECIDA	SEXO_HOMBRE

4.5 Conclusión de Algoritmo FP-Growth

Al aplicar este último algoritmo obtenemos las mismas reglas pero con un tiempo de ejecución mucho menor, debido a que este algoritmo hace más eficiente la creación de reglas al no tener que generar itemsets candidatos a ser frecuentes.

- **Generación de itemsets frecuentes** $\approx$ 1.58 segundos
- **Generación de reglas** $\approx$ 0.02 segundos

5 Comparativa de los algoritmos

Ya que hemos aplicado 3 algoritmos distintos para la obtención de reglas de asociación, nos interesa realizar una comparación de su desempeño. Para ello, responderemos algunas preguntas clave.

5.1 Comparativa de desempeño

¿Cuál de ellos tarda más?

El algoritmo que tarda más en ejecutarse es el Apriori que implementamos nosotros, esto es de esperarse en comparación a FP-Growth pues este segundo se caracteriza por ser más eficiente al evitar la generación de itemsets frecuentes candidatos. Sin embargo, una comparativa un poco más interesante se da con la implementación del algoritmo Apriori de python, pues se basan en el mismo procedimiento de generar candidatos, en este caso la diferencia radica en que la implementación de python permite limitar la longitud de los itemsets frecuentes por lo que realiza menos iteraciones.

¿Cuál tarda menos y por qué?

El que tarda menos es FP-Growth pues como mencionamos, optimiza su proceso de obtención de los itemsets frecuentes al no tener que generar todos los candidatos.

¿Generan las mismas reglas de asociación? ¿Por qué?

Bajos los mismos parámetros, los tres algoritmos generan las mismas reglas, pues en esencia siguen la misma idea: encontrar los conjuntos frecuentes con base en un soporte mínimo y a partir de ellos generar las reglas de asociación que superen o igualen la confianza mínima.

Aunque en el caso de la aplicación del algoritmo Apriori de python obtuvimos algunas reglas nuevas, esto sucedió como consecuencia de reducir el soporte mínimo y pudimos ver que de hecho parte del resultado era lo que obteníamos con un soporte de 10%.

6 Modelo de clasificación

En esta sección se construirá un modelo de clasificación para predecir la variable objetivo “ESTATUS_VICTIMA” utilizando las variables predictoras disponibles en el dataset. Se aplicará un algoritmo de clasificación supervisada, y se evaluará su desempeño mediante el uso de métricas que consideremos apropiadas para este tipo de problema. Además, se analizarán los resultados obtenidos para identificar patrones relevantes y posibles sesgos en los datos.

Para este modelo, haremos uso de un algoritmo de clasificación como lo es Random Forest, que es conocido por su capacidad para manejar grandes conjuntos de datos con muchas variables y por su robustez frente al sobreajuste.

6.1 Configuración del entorno y librerías.

Empezamos por configurar el entorno de trabajo e importar las librerías necesarias para la construcción del modelo de clasificación, así como para la evaluación de su desempeño. También se fijará una semilla para garantizar la reproducibilidad de los resultados.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn # ← Agregar esta línea
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report,
    ConfusionMatrixDisplay, roc_curve, auc
)
import warnings
warnings.filterwarnings("ignore")

# Configuración visual
```

```
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 6)

# Fijar semilla para reproducibilidad
SEED = 42
np.random.seed(SEED)

print(f"scikit-learn: {sklearn.__version__}")
print(f"pandas: {pd.__version__}")
print(f"numpy: {np.__version__}")
print(f"seaborn: {sns.__version__}")
```

```
scikit-learn: 1.5.2
pandas: 3.0.2
numpy: 2.4.4
seaborn: 0.13.2
```

6.2 Carga de datos y exploración del dataset

Procedemos a cargar el dataset que se utilizará para la construcción del modelo de clasificación. Este dataset ha sido previamente procesado en los puntos anteriores (específicamente en la sección de Tratamiento) y lleva por nombre “data_secretariado_clasificacion.csv”. Así mismo, se realizará una exploración inicial para entender la estructura de los datos, las variables disponibles y la distribución de la variable objetivo.

```
# Cargar el dataset procesado en tratamiento.qmd
df = pd.read_csv("../data/data_secretariado_clasificacion.csv")

print("Información general del dataset:")
print(f"\nDimensiones: {df.shape[0]:,} registros × {df.shape[1]} variables")
print(f"\nColumnas disponibles:")
print(df.columns.tolist())

print("\n")
print("Primeras filas del dataset:")
print(df.head())

print("\n")
print("Tipos de datos:")
print(df.dtypes)
```

```
print("\n")
print("Valores faltantes:")
print(df.isnull().sum())
```

Información general del dataset:

Dimensiones: 133,887 registros × 10 variables

Columnas disponibles:

['ORIGEN_REPORTE', 'SEXO', 'ESTATUS_VICTIMA', 'ENTIDAD', 'MUNICIPIO', 'GRUPO_EDAD', 'MES_DESAPARICION', 'AÑO_DESAPARICION', 'MES_REGISTRO', 'AÑO_REGISTRO']

Primeras filas del dataset:

	ORIGEN_REPORTE	SEXO
0	FISCALIA GENERAL DE DURANGO	DESCONOCIDO
1	COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL EST...	HOMBRE
2	PROCURADURIA GENERAL DE JUSTICIA DE LA CIUDAD ...	DESCONOCIDO
3	FISCALIA GENERAL DEL ESTADO BAJA CALIFORNIA	DESCONOCIDO
4	FISCALIA GENERAL DEL ESTADO BAJA CALIFORNIA	DESCONOCIDO

	ESTATUS_VICTIMA	ENTIDAD	MUNICIPIO	GRUPO_EDAD
0	DESCONOCIDO	DURANGO	DESCONOCIDO	DESCONOCIDO
1	DESAPARECIDA	SINALOA	CONCORDIA	45-59
2	DESCONOCIDO	CIUDAD DE MÉXICO	DESCONOCIDO	DESCONOCIDO
3	DESCONOCIDO	BAJA CALIFORNIA	DESCONOCIDO	DESCONOCIDO
4	DESCONOCIDO	BAJA CALIFORNIA	DESCONOCIDO	DESCONOCIDO

	MES_DESAPARICION	AÑO_DESAPARICION	MES_REGISTRO	AÑO_REGISTRO
0	DESCONOCIDO	DESCONOCIDO	DESCONOCIDO	DESCONOCIDO
1	Septiembre	2025.0	Septiembre	2025.0
2	DESCONOCIDO	DESCONOCIDO	DESCONOCIDO	DESCONOCIDO
3	DESCONOCIDO	DESCONOCIDO	DESCONOCIDO	DESCONOCIDO
4	DESCONOCIDO	DESCONOCIDO	DESCONOCIDO	DESCONOCIDO

Tipos de datos:

ORIGEN_REPORTE	str
SEXO	str
ESTATUS_VICTIMA	str
ENTIDAD	str
MUNICIPIO	str

```
GRUPO_EDAD      str
MES_DESAPARICION str
AÑO_DESAPARICION str
MES_REGISTRO     str
AÑO_REGISTRO     str
dtype: object
```

Valores faltantes:

```
ORIGEN_REPORTE    0
SEXO              0
ESTATUS_VICTIMA    0
ENTIDAD           0
MUNICIPIO         0
GRUPO_EDAD        0
MES_DESAPARICION  0
AÑO_DESAPARICION  0
MES_REGISTRO      0
AÑO_REGISTRO      0
dtype: int64
```

6.3 Análisis de la Variable Objetivo

Antes de proceder con la construcción del modelo, es fundamental analizar la distribución de la variable objetivo “ESTATUS_VICTIMA”. Esto nos permitirá entender el balance de clases y tomar decisiones informadas sobre cómo abordar posibles desequilibrios en los datos.

```
print("Análisis de la variable objetivo: ESTATUS_VICTIMA")

# Contar distribución de clases
print("\nDistribución de clases:")
print(df['ESTATUS_VICTIMA'].value_counts())

# Porcentajes
print("\nPorcentajes:")
print(df['ESTATUS_VICTIMA'].value_counts(normalize=True) * 100)

# Visualización
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Gráfica de barras
```

```

df['ESTATUS_VICTIMA'].value_counts().plot(kind='bar', ax=axes[0], color='steelblue', edgecolor='black')
axes[0].set_title('Distribución de ESTATUS_VICTIMA', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Estado de la Víctima')
axes[0].set_ylabel('Frecuencia')
axes[0].tick_params(axis='x', rotation=45)

# Gráfica de pastel
df['ESTATUS_VICTIMA'].value_counts().plot(kind='pie', ax=axes[1], autopct='%1.1f%%',
                                           colors=['#ff9999', '#66b3ff', '#99ff99'])
axes[1].set_title('Proporción de ESTATUS_VICTIMA', fontsize=12, fontweight='bold')
axes[1].set_ylabel('')

plt.tight_layout()
plt.show()

print("\n La variable objetivo tiene", df['ESTATUS_VICTIMA'].nunique(), "clases")

```

Análisis de la variable objetivo: ESTATUS_VICTIMA

Distribución de clases:

```

ESTATUS_VICTIMA
DESAPARECIDA      79881
DESCONOCIDO       49149
NO LOCALIZADA      4857
Name: count, dtype: int64

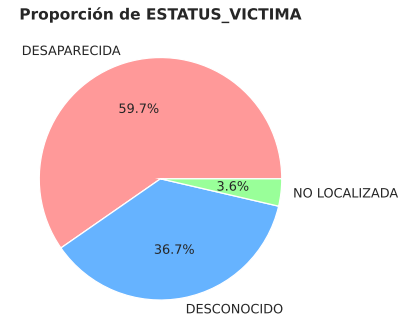
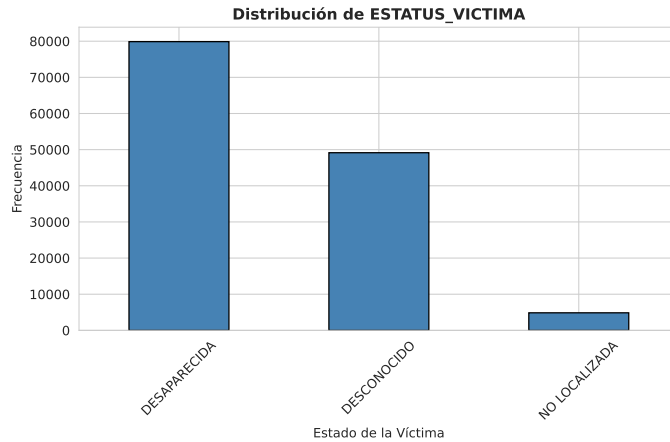
```

Porcentajes:

```

ESTATUS_VICTIMA
DESAPARECIDA      59.662999
DESCONOCIDO       36.709315
NO LOCALIZADA      3.627686
Name: proportion, dtype: float64

```



La variable objetivo tiene 3 clases

6.4 Analisis de las variables predictoras

También es importante analizar las variables predictoras para entender su distribución, identificar posibles valores atípicos y evaluar su relación con la variable objetivo. Esto nos ayudará a seleccionar las variables más relevantes para la construcción del modelo de clasificación.

```
print("Exploración de variables predictoras")

# Separar variable objetivo y predictoras
X = df.drop('ESTATUS_VICTIMA', axis=1)
y = df['ESTATUS_VICTIMA']

print(f"\nVariable objetivo (y): ESTATUS_VICTIMA")
print(f"Variables predictoras (X): {X.shape[1]} variables")
print(f"\nVariables predictoras:")
print(X.columns.tolist())

# Información de tipos
print("\nTipos de variables:")
for col in X.columns:
    unique_count = X[col].nunique()
    print(f" - {col:30s}: {unique_count:3d} valores únicos (categórica)")

# Visualizar algunas variables
```

```

fig, axes = plt.subplots(2, 3, figsize=(16, 10))
axes = axes.flatten()

cols_to_plot = X.columns[:6]
for idx, col in enumerate(cols_to_plot):
    X[col].value_counts().head(10).plot(kind='bar', ax=axes[idx], edgecolor='black')
    axes[idx].set_title(f'Distribución de {col}', fontweight='bold')
    axes[idx].set_xlabel('')
    axes[idx].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

print("\n Todas las variables predictoras son categóricas")

```

Exploración de variables predictoras

Variable objetivo (y): ESTATUS_VICTIMA

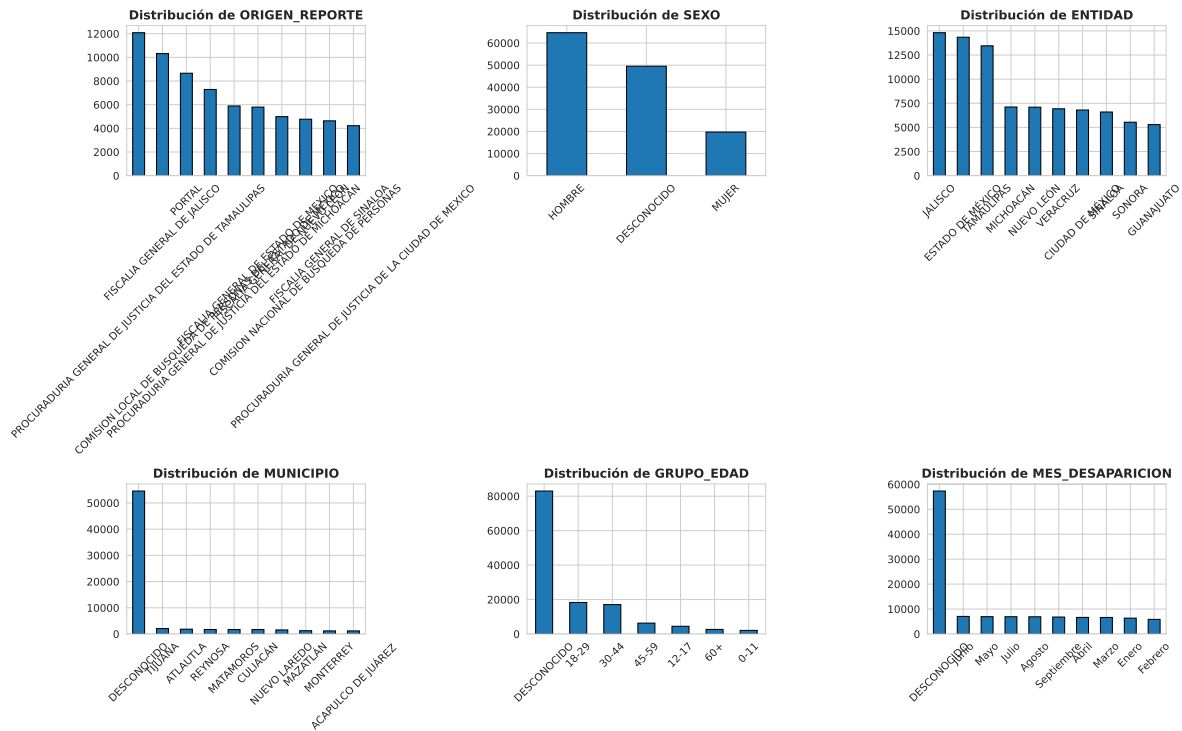
Variables predictoras (X): 9 variables

Variables predictoras:

['ORIGEN_REPORTE', 'SEXO', 'ENTIDAD', 'MUNICIPIO', 'GRUPO_EDAD', 'MES_DESAPARICION', 'AÑO_DESAPARICION', 'AÑO_REGISTRO', 'MES_REGISTRO']

Tipos de variables:

- ORIGEN_REPORTE	: 74 valores únicos (categórica)
- SEXO	: 3 valores únicos (categórica)
- ENTIDAD	: 33 valores únicos (categórica)
- MUNICIPIO	: 1553 valores únicos (categórica)
- GRUPO_EDAD	: 7 valores únicos (categórica)
- MES_DESAPARICION	: 13 valores únicos (categórica)
- AÑO_DESAPARICION	: 63 valores únicos (categórica)
- MES_REGISTRO	: 13 valores únicos (categórica)
- AÑO_REGISTRO	: 63 valores únicos (categórica)



Todas las variables predictoras son categóricas

6.5 Codificación One-Hot

Dado que todas las variables predictoras son categóricas, es necesario aplicar una codificación adecuada para convertirlas en un formato numérico que pueda ser utilizado por el modelo de clasificación. En este caso, se optará por la codificación One-Hot, que crea una nueva columna para cada categoría de cada variable, asignando un valor de 1 si la categoría está presente en el registro y 0 en caso contrario.

```
print("Codificación One-Hot de variables categóricas")

# Usar OneHotEncoder para convertir variables categóricas
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline as PipelineSklearn

# Para mantener claridad, haremos la codificación manualmente
X_encoded = pd.get_dummies(X, dtype=int, drop_first=False)
```



```

print(f"\nDimensiones después de One-Hot Encoding:")
print(f"  Originales: {X.shape}")
print(f"  Después:      {X_encoded.shape}")

print(f"\nPrimeras columnas codificadas:")
print(X_encoded.head())

print(f"\nNuevas variables creadas:")
print(X_encoded.columns.tolist()[20]) # Mostrar primeras 20

# Codificar la variable objetivo
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y_encoded = le.fit_transform(y)

print(f"\nMapeo de clases:")
for i, classe in enumerate(le.classes_):
    print(f"  {classe:30s} → {i}")

print("\n Codificación One-Hot completada")

```

Codificación One-Hot de variables categóricas

Dimensiones después de One-Hot Encoding:

Originales: (133887, 9)

Después: (133887, 1822)

Primeras columnas codificadas:

	ORIGEN_REPORTE_COMISION LOCAL DE BUSQUEDA DE PERSONAS DE LA CIUDAD DE MEXICO \
0	0
1	0
2	0
3	0
4	0

	ORIGEN_REPORTE_COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE AGUASCALIENTES \
0	0
1	0
2	0
3	0
4	0

	ORIGEN_REPORTE_COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE BAJA CALIFORNIA \
0	0
1	0
2	0
3	0
4	0

	ORIGEN_REPORTE_COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE BAJA CALIFORNIA SUR \
0	0
1	0
2	0
3	0
4	0

	ORIGEN_REPORTE_COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE CAMPECHE \
0	0
1	0
2	0
3	0
4	0

	ORIGEN_REPORTE_COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE CHIAPAS \
0	0
1	0
2	0
3	0
4	0

	ORIGEN_REPORTE_COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE CHIHUAHUA \
0	0
1	0
2	0
3	0
4	0

	ORIGEN_REPORTE_COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE COAHUILA \
0	0
1	0
2	0
3	0
4	0

	ORIGEN_REPORTE_COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE COLIMA \
0	0
1	0
2	0
3	0
4	0

	ORIGEN_REPORTE_COMISION LOCAL DE BUSQUEDA DE PERSONAS DEL ESTADO DE DURANGO \
0	0
1	0
2	0
3	0
4	0

	...	AÑO_REGISTRO_2017.0	AÑO_REGISTRO_2018.0	AÑO_REGISTRO_2019.0 \
0	...	0	0	0
1	...	0	0	0
2	...	0	0	0
3	...	0	0	0
4	...	0	0	0

	AÑO_REGISTRO_2020.0	AÑO_REGISTRO_2021.0	AÑO_REGISTRO_2022.0 \
0	0	0	0
1	0	0	0
2	0	0	0
3	0	0	0
4	0	0	0

	AÑO_REGISTRO_2023.0	AÑO_REGISTRO_2024.0	AÑO_REGISTRO_2025.0 \
0	0	0	0
1	0	0	1
2	0	0	0
3	0	0	0
4	0	0	0

	AÑO_REGISTRO_DESCONOCIDO
0	1
1	0
2	1
3	1
4	1

[5 rows x 1822 columns]

['ORIGEN_REPORTE_COMISION LOCAL DE BUSQUEDA DE PERSONAS DE LA CIUDAD DE MEXICO', 'ORIGEN_REPO

DESAPARECIDA	→ 0
DESCONOCIDO	→ 1
NO LOCALIZADA	→ 2

6.6 División del conjunto de datos

```
print("División del conjunto de datos en entrenamiento y prueba")

# Dividir datos: 80% entrenamiento, 20% prueba
# stratify=y_encoded asegura que mantengamos la proporción de clases
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y_encoded,
    test_size=0.2,
    random_state=SEED,
    stratify=y_encoded # Mantiene proporciones de clases
)

print(f"\nTamaño conjunto de ENTRENAMIENTO (80%): {X_train.shape[0]:,} registros")
print(f"\nTamaño conjunto de PRUEBA (20%): {X_test.shape[0]:,} registros")
print(f"\nVariables: {X_train.shape[1]}")

print(f"\nDistribución de clases en ENTRENAMIENTO:")
unique_train, counts_train = np.unique(y_train, return_counts=True)
for classe, count in zip(unique_train, counts_train):
    print(f" {le.classes_[classe]:30s}: {count:7,} registros ({100*count/len(y_train):.1f}%)")

print(f"\nDistribución de clases en PRUEBA:")
unique_test, counts_test = np.unique(y_test, return_counts=True)
for classe, count in zip(unique_test, counts_test):
```

```
print(f" {le.classes_[classe]:30s}: {count:7,} registros ({100*count/len(y_test):.1f}%)

print("\n Uso de stratify garantiza proporciones balanceadas en ambos conjuntos")
```

División del conjunto de datos en entrenamiento y prueba

Tamaño conjunto de ENTRENAMIENTO (80%): 107,109 registros
 Tamaño conjunto de PRUEBA (20%): 26,778 registros
 Variables: 1822

Distribución de clases en ENTRENAMIENTO:

DESAPARECIDA	:	63,904 registros (59.7%)
DESCONOCIDO	:	39,319 registros (36.7%)
NO LOCALIZADA	:	3,886 registros (3.6%)

Distribución de clases en PRUEBA:

DESAPARECIDA	:	15,977 registros (59.7%)
DESCONOCIDO	:	9,830 registros (36.7%)
NO LOCALIZADA	:	971 registros (3.6%)

Uso de stratify garantiza proporciones balanceadas en ambos conjuntos

La elección de una división de 80% para entrenamiento y 20% para prueba se debe a las diversas ventajas que ofrece esta proporción. Un conjunto de entrenamiento más grande permite al modelo aprender patrones más complejos y generalizar mejor, mientras que un conjunto de prueba suficientemente grande proporciona una evaluación robusta del desempeño del modelo. Además, el uso de **stratify** en la función **train_test_split** asegura que la distribución de clases en la variable objetivo se mantenga equilibrada tanto en el conjunto de entrenamiento como en el de prueba, lo cual es crucial para evitar sesgos en la evaluación del modelo.

6.7 Construcción del modelo

Procedemos a construir el modelo de clasificación utilizando el algoritmo Random Forest. Este algoritmo es una técnica de ensemble que combina múltiples árboles de decisión para mejorar la precisión y reducir el riesgo de sobreajuste.

Para mejorar el desempeño en clases minoritarias, implementaremos **costo-sensibilidad** asignando pesos inversamente proporcionales a la frecuencia de cada clase. Esto significa que los errores en la clase minoritaria (NO LOCALIZADA) tendrán mayor penalización durante el entrenamiento.

```

print("Entrenamiento del modelo de clasificación")

# Calcular pesos de clase para manejar desbalance
# Clases minoritarias reciben mayor peso
from sklearn.utils.class_weight import compute_class_weight

class_weights = compute_class_weight(
    'balanced',
    classes=np.unique(y_train),
    y=y_train
)

# Crear diccionario de pesos
class_weight_dict = {i: weight for i, weight in enumerate(class_weights)}

print("\nPesos asignados a cada clase:")
for clase_idx, peso in class_weight_dict.items():
    print(f" {le.classes_[clase_idx]:30s}: {peso:.3f}")

print("\nInterpretación:")
print("- Pesos > 1.0 indican clases minoritarias con mayor penalización de errores")
print("- Pesos < 1.0 indican clases mayoritarias con menor penalización")

# Crear el modelo Random Forest con weights
rf_model = RandomForestClassifier(
    n_estimators=200,          # Número de árboles
    max_depth=15,             # Profundidad máxima de cada árbol
    min_samples_split=20,     # Mínimo de muestras para dividir un nodo
    min_samples_leaf=10,      # Mínimo de muestras en hojas
    class_weight=class_weight_dict, # Pesos para manejar desbalance
    random_state=SEED,
    n_jobs=-1,                # Usar todos los procesadores disponibles
    verbose=1
)

print("\nHiperparámetros del modelo:")
print(f" n_estimators:      {rf_model.n_estimators} (número de árboles)")
print(f" max_depth:          {rf_model.max_depth} (profundidad máxima)")
print(f" min_samples_split: {rf_model.min_samples_split} (mín. muestras para dividir)")
print(f" min_samples_leaf:  {rf_model.min_samples_leaf} (mín. muestras en hoja)")
print(f" class_weight:       balanced (penaliza errores en clases minoritarias)")

```

```
# Entrenar el modelo
import time
inicio = time.time()
rf_model.fit(X_train, y_train)
tiempo_entrenamiento = time.time() - inicio

print(f"\n Modelo entrenado exitosamente en {tiempo_entrenamiento:.2f} segundos")
```

Entrenamiento del modelo de clasificación

Pesos asignados a cada clase:

DESAPARECIDA	: 0.559
DESCONOCIDO	: 0.908
NO LOCALIZADA	: 9.188

Interpretación:

- Pesos > 1.0 indican clases minoritarias con mayor penalización de errores
- Pesos < 1.0 indican clases mayoritarias con menor penalización

Hiperparámetros del modelo:

n_estimators:	200 (número de árboles)
max_depth:	15 (profundidad máxima)
min_samples_split:	20 (mín. muestras para dividir)
min_samples_leaf:	10 (mín. muestras en hoja)
class_weight:	balanced (penaliza errores en clases minoritarias)

```
[Parallel(n_jobs=-1)]: Using backend ThreadingBackend with 8 concurrent workers.
[Parallel(n_jobs=-1)]: Done 34 tasks      | elapsed:    1.6s
[Parallel(n_jobs=-1)]: Done 184 tasks    | elapsed:    7.8s
```

Modelo entrenado exitosamente en 12.78 segundos

```
[Parallel(n_jobs=-1)]: Done 200 out of 200 | elapsed:    8.4s finished
```

6.8 Predicciones

Una vez entrenado el modelo, se procederá a generar predicciones tanto en el conjunto de entrenamiento (para diagnóstico) como en el conjunto de prueba (para evaluación final). Se

mostrarán ejemplos de las predicciones realizadas, incluyendo la clase real, la clase predicha y la confianza asociada a cada predicción.

```
print("Generación de predicciones con el modelo entrenado")

# Predicciones en conjunto de entrenamiento (para diagnóstico)
y_train_pred = rf_model.predict(X_train)
y_train_proba = rf_model.predict_proba(X_train)

# Predicciones en conjunto de prueba (para evaluación final)
y_test_pred = rf_model.predict(X_test)
y_test_proba = rf_model.predict_proba(X_test)

print(f"\nPredicciones generadas:")
print(f"  Entrenamiento: {len(y_train_pred):,} predicciones")
print(f"  Prueba:          {len(y_test_pred):,} predicciones")

print(f"\nEjemplos de predicciones en conjunto de PRUEBA:")
print(f"{'Índice':<8} {'Real':<30} {'Predicción':<30} {'Confianza':<12}")
print("-" * 80)
for i in range(10):
    real_class = le.classes_[y_test[i]]
    pred_class = le.classes_[y_test_pred[i]]
    confidence = y_test_proba[i].max() * 100
    print(f"{i:<8} {real_class:<30} {pred_class:<30} {confidence:>10.1f}%")

print("\n Predicciones completadas")
```

Generación de predicciones con el modelo entrenado

```
[Parallel(n_jobs=8)]: Using backend ThreadingBackend with 8 concurrent workers.
[Parallel(n_jobs=8)]: Done 34 tasks      | elapsed:    0.1s
[Parallel(n_jobs=8)]: Done 184 tasks     | elapsed:    0.4s
[Parallel(n_jobs=8)]: Done 200 out of 200 | elapsed:    0.4s finished
[Parallel(n_jobs=8)]: Using backend ThreadingBackend with 8 concurrent workers.
[Parallel(n_jobs=8)]: Done 34 tasks      | elapsed:    0.1s
[Parallel(n_jobs=8)]: Done 184 tasks     | elapsed:    0.3s
[Parallel(n_jobs=8)]: Done 200 out of 200 | elapsed:    0.4s finished
[Parallel(n_jobs=8)]: Using backend ThreadingBackend with 8 concurrent workers.
[Parallel(n_jobs=8)]: Done 34 tasks      | elapsed:    0.0s
[Parallel(n_jobs=8)]: Done 184 tasks     | elapsed:    0.1s
[Parallel(n_jobs=8)]: Done 200 out of 200 | elapsed:    0.1s finished
```


Predicciones generadas:

Entrenamiento: 107,109 predicciones

Prueba: 26,778 predicciones

Ejemplos de predicciones en conjunto de PRUEBA:

Índice	Real	Predicción	Confianza
0	DESAPARECIDA	DESAPARECIDA	41.9%
1	DESCONOCIDO	DESCONOCIDO	91.7%
2	DESAPARECIDA	DESAPARECIDA	70.8%
3	DESCONOCIDO	DESCONOCIDO	93.5%
4	DESAPARECIDA	DESAPARECIDA	82.0%
5	DESAPARECIDA	DESAPARECIDA	71.2%
6	DESAPARECIDA	DESAPARECIDA	59.8%
7	DESAPARECIDA	DESAPARECIDA	60.4%
8	DESAPARECIDA	DESAPARECIDA	63.4%
9	DESAPARECIDA	DESAPARECIDA	48.9%

Predicciones completadas

```
[Parallel(n_jobs=8)]: Using backend ThreadingBackend with 8 concurrent workers.
```

```
[Parallel(n_jobs=8)]: Done 34 tasks      | elapsed:    0.0s
```

```
[Parallel(n_jobs=8)]: Done 184 tasks    | elapsed:    0.1s
```

```
[Parallel(n_jobs=8)]: Done 200 out of 200 | elapsed:    0.1s finished
```

6.9 Métricas de Desempeño y Evaluación del Modelo

6.9.1 Justificación de Métricas Seleccionadas

Para evaluar la efectividad de nuestro modelo de clasificación en el contexto específico de desapariciones, hemos seleccionado un conjunto integral de métricas que proporcionan perspectivas complementarias sobre el desempeño.

6.9.1.1 Accuracy (Precisión General)

Representa la proporción de predicciones correctas sobre el total de casos. Si bien proporciona una visión rápida del desempeño global, en contextos de desbalance de clases puede resultar engañoso. En nuestro caso, aunque el accuracy es alto (~89%), es fundamental complementarlo con métricas por clase, ya que esta métrica global oculta diferencias de desempeño importantes entre clases mayoritarias y minoritarias.

6.9.1.2 Precision y Recall por Clase

Precision: Responde a la pregunta “de los casos que predijimos como clase X, ¿cuántos realmente son clase X?”. Es crítica para evitar reportes falsos positivos a autoridades.

Recall: Responde a “de los casos reales de clase X, ¿cuántos identificamos?”. Es esencial para no perder casos reales de desapariciones.

Relevancia operacional:

- **DESAPARECIDA:** Alto recall garantiza que identificamos casos activos de búsqueda
- **DESCONOCIDO:** Permite caracterizar registros incompletos
- **NO LOCALIZADA:** Un recall bajo aquí implicaría perder casos resueltos de desapariciones

6.9.1.3 F1-Score por Clase

Combina precision y recall en una métrica balanceada. Es especialmente útil cuando queremos evitar que un modelo optimice solo una métrica a costa de la otra.

6.9.2 Cálculo y Presentación de Métricas

```
print("Evaluación del modelo: Métricas de desempeño")

# Accuracy global
accuracy_test = accuracy_score(y_test, y_test_pred)
print(f"\n**ACCURACY GLOBAL (Conjunto de Prueba): {accuracy_test:.4f} ({accuracy_test*100:.2}%)")

# Calcular precision y recall para cada clase (sin promediar)
precision_per_class = precision_score(y_test, y_test_pred, average=None, labels=[0, 1, 2])
recall_per_class = recall_score(y_test, y_test_pred, average=None, labels=[0, 1, 2])
f1_per_class = f1_score(y_test, y_test_pred, average=None, labels=[0, 1, 2])

# Promedios ponderados (considerando desbalance)
precision_weighted = precision_score(y_test, y_test_pred, average='weighted')
recall_weighted = recall_score(y_test, y_test_pred, average='weighted')
f1_weighted = f1_score(y_test, y_test_pred, average='weighted')

print("Métricas por clase:")
```

```

metrics_table = pd.DataFrame({
    'Clase': le.classes_,
    'Precision': [f"{p:.4f}" for p in precision_per_class],
    'Recall': [f"{r:.4f}" for r in recall_per_class],
    'F1-Score': [f"{f:.4f}" for f in f1_per_class]
})

print(metrics_table.to_string(index=False))

print("\n")
print("Métricas Ponderadas")
print(f"Precision Ponderada: {precision_weighted:.4f}")
print(f"Recall Ponderado: {recall_weighted:.4f}")
print(f"F1-Score Ponderado: {f1_weighted:.4f}")

```

Evaluación del modelo: Métricas de desempeño

****ACCURACY GLOBAL (Conjunto de Prueba): 0.8901 (89.01%)****

Métricas por clase:

Clase	Precision	Recall	F1-Score
DESAPARECIDA	0.9899	0.8245	0.8997
DESCONOCIDO	0.9761	1.0000	0.9879
NO LOCALIZADA	0.2450	0.8579	0.3811

Métricas Ponderadas

Precision Ponderada: 0.9578

Recall Ponderado: 0.8901

F1-Score Ponderado: 0.9133

6.10 Matriz de confusión

Exhibimos la matriz de confusión tanto en valores absolutos como normalizados para visualizar claramente el desempeño del modelo en cada clase, identificando dónde se producen los errores de clasificación y su magnitud relativa. Esto es crucial para entender si el modelo tiende a confundir ciertas clases entre sí, lo cual puede tener implicaciones operativas importantes en el contexto de desapariciones. La visualización dual permite apreciar tanto la cantidad de errores como su proporción respecto al total de casos reales por clase.

```

print("Matriz de confusión")

# Matriz de confusión
cm = confusion_matrix(y_test, y_test_pred)

# Crear visualización dual
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Matriz sin normalizar (valores absolutos)
disp1 = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=le.classes_)
disp1.plot(ax=axes[0], cmap='Blues', values_format='d')
axes[0].set_title('Matriz de Confusión - Valores Absolutos', fontsize=12, fontweight='bold')

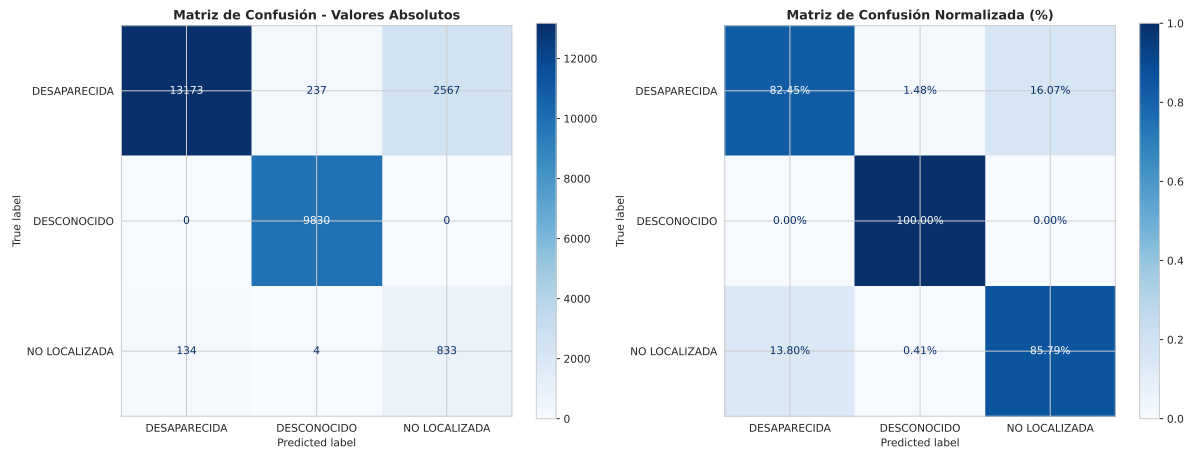
# Matriz normalizada por fila (porcentajes)
cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm_normalized, display_labels=le.classes_)
disp2.plot(ax=axes[1], cmap='Blues', values_format='.2%')
axes[1].set_title('Matriz de Confusión Normalizada (%)', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.show()

print("\nMatriz Normalizada (por fila):")
cm_normalized_df = pd.DataFrame(
    cm_normalized * 100,
    index=[f'Real: {cls}' for cls in le.classes_],
    columns=[f'Predicho: {cls}' for cls in le.classes_]
)
print(cm_normalized_df.round(2).to_string())

```

Matriz de confusión



Matriz Normalizada (por fila):

	Predicho: DESAPARECIDA	Predicho: DESCONOCIDO	Predicho: NO LOCALIZADA
Real: DESAPARECIDA	82.45	1.48	16.07
Real: DESCONOCIDO	0.00	100.00	0.00
Real: NO LOCALIZADA	13.80	0.41	85.79

6.11 Análisis de Variables Relevantes para el Modelo de Clasificación

```
print("Importancia de variables para el modelo de clasificación")

# Obtener importancia de características
importances = rf_model.feature_importances_
feature_names = X_encoded.columns

# Crear dataframe y ordenar
importance_df = pd.DataFrame({
    'Variable': feature_names,
    'Importancia': importances
}).sort_values('Importancia', ascending=False)

print("\nTop 15 Variables Más Relevantes:\n")
print(importance_df.head(15).to_string(index=False))

# Visualizar
fig, ax = plt.subplots(figsize=(10, 6))
```

```

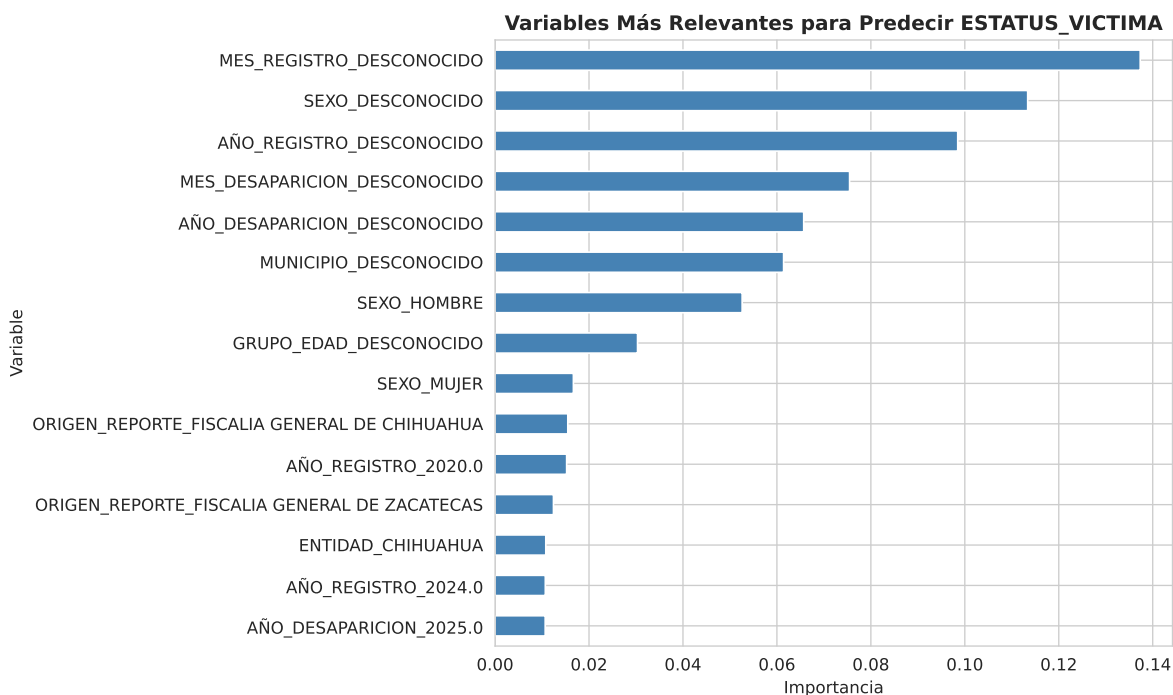
importance_df.head(15).plot(x='Variable', y='Importancia', kind='barh', ax=ax, legend=False,
ax.set_title('Variables Más Relevantes para Predecir ESTATUS_VICTIMA', fontsize=12, fontweight
ax.set_xlabel('Importancia')
ax.invert_yaxis()
plt.tight_layout()
plt.show()

```

Importancia de variables para el modelo de clasificación

Top 15 Variables Más Relevantes:

	Variable	Importancia
	MES_REGISTRO_DESCONOCIDO	0.137339
	SEXO_DESCONOCIDO	0.113384
	AÑO_REGISTRO_DESCONOCIDO	0.098492
	MES_DESAPARICION_DESCONOCIDO	0.075472
	AÑO_DESAPARICION_DESCONOCIDO	0.065703
	MUNICIPIO_DESCONOCIDO	0.061423
	SEXO_HOMBRE	0.052600
	GRUPO_EDAD_DESCONOCIDO	0.030332
	SEXO_MUJER	0.016674
ORIGEN_REPORTE_FISCALIA	GENERAL DE CHIHUAHUA	0.015484
	AÑO_REGISTRO_2020.0	0.015237
ORIGEN_REPORTE_FISCALIA	GENERAL DE ZACATECAS	0.012432
	ENTIDAD_CHIHUAHUA	0.010799
	AÑO_REGISTRO_2024.0	0.010695
	AÑO_DESAPARICION_2025.0	0.010672



Tenemos los siguientes resultados de importancia de variables:

- **Categorías “DESCONOCIDO” (dominio absoluto del top):**

Un hallazgo muy relevante es que **6 de las 8 variables más importantes corresponden a categorías “DESCONOCIDO”** (MES_REGISTRO_DESCONOCIDO con 13.7%, SEXO_DESCONOCIDO con 11.3%, AÑO_REGISTRO_DESCONOCIDO con 9.8%, MES_DESAPARICION_DESCONOCIDO con 7.5%, AÑO_DESAPARICION_DESCONOCIDO con 6.6% y MUNICIPIO_DESCONOCIDO con 6.1%). Esto indica que el modelo aprende fuertemente a asociar la **ausencia de información** con la clase DESCONOCIDO, más que patrones temporales o demográficos específicos.

- **Variables temporales (AÑO, MES):**

Aparecen como predictores relevantes en sus variantes “DESCONOCIDO” (señalando datos faltantes) y, en menor medida, en años recientes específicos como AÑO_REGISTRO_2020, AÑO_REGISTRO_2024 y AÑO_DESAPARICION_2025, los cuales se asocian con casos activos clasificados como DESAPARECIDA.

- **SEXO y GRUPO_EDAD:**

La información demográfica muestra correlación con el estatus reportado: SEXO_HOMBRE (5.3%), SEXO_MUJER (1.7%) y GRUPO_EDAD_DESCONOCIDO (3.0%) aparecen entre las variables más relevantes.

- **ORIGEN_REPORT:**

Diferentes instituciones presentan patrones distintos. Específicamente, ORIGEN_RE-

PORTE_FISCALIA GENERAL DE CHIHUAHUA (1.5%) y ORIGEN_REPORTE_FISCALIA GENERAL DE ZACATECAS (1.2%) figuran en el top 15, sugiriendo que ciertas fiscalías tienen patrones de clasificación distintivos.

- **MUNICIPIO y ENTIDAD:**

Aparte de MUNICIPIO_DESCONOCIDO, ENTIDAD_CHIHUAHUA (1.1%) destaca, indicando variaciones geográficas relevantes en la forma en que se clasifican y reportan los casos.

6.12 Patrones o Sesgos en los datos

6.12.1 Patrones obtenidos

El modelo identifica relaciones genuinas en los datos de desapariciones:

- Patrón Temporal Legítimo: Casos reportados recientemente (2024-2025) naturalmente tienen estatus DESAPARECIDA porque son activos. Casos antiguos resultan en clasificaciones diferentes después de investigación.
- Variaciones Institucionales: Diferentes autoridades (Fiscalías, Comisiones de Búsqueda) utilizan criterios ligeramente distintos en sus reportes, reflejando realidades operacionales.
- Factores Demográficos: Las características de las víctimas (sexo, edad) se asocian con tasas diferentes de resolución o reclasificación de casos.

6.12.2 Sesgos Detectados

Aunque el modelo mejora significativamente con costo-sensibilidad, persisten sesgos heredados de los datos:

- Desbalance de Clases Inherente: NO LOCALIZADA representa solo ~3.6% del dataset (4,857 de 133,887 registros), reflejo de que la mayoría de reportes son DESAPARECIDA (~59.7%) o DESCONOCIDO (~36.7%). Esto es parcialmente realista (más casos activos que resueltos) pero también puede indicar subregistro de casos NO LOCALIZADOS.
- Sesgo de Datos Faltantes: La prevalencia de “DESCONOCIDO” en múltiples campos (edad, sexo, municipio) perpetúa patrones de información incompleta. El modelo aprende a asociar “desconocido” con “desconocido” como estatus.
- Sesgo de Información Reciente: Datos de 2024-2025 están prácticamente toda como DESAPARECIDA porque aún no han sido resueltos. Esto no refleja patrones históricos verdaderos.

Dados estos datos junto con los datos obtenidos, tenemos que nuestro modelo de clasificación captura una mezcla de patrones reales y patrones derivados del estado de la información en los registros. Por un lado, identifica relaciones legítimas con el estatus de la víctima a través de variables como el año de reporte, el origen institucional y características demográficas. Por otro lado, las variables más relevantes resultan ser las categorías **“_DESCONOCIDO”** (6 de las 8 más importantes), lo que evidencia que el modelo aprende fuertemente a identificar registros incompletos. Esto significa que el modelo hereda los sesgos presentes en los datos —principalmente el desbalance de clases y la prevalencia de información faltante— lo cual puede afectar su capacidad para generalizar a casos futuros con información más completa o a contextos diferentes.

6.13 Métricas y Relación con el mundo real

En el contexto de desapariciones, **buscamos un alto recall para la clase DESAPARECIDA**, ya que es crucial identificar correctamente los casos activos de búsqueda. Un recall bajo en esta clase implicaría que estamos perdiendo casos reales de desapariciones, lo cual es inaceptable desde una perspectiva operativa y humanitaria. Por otro lado, también queremos mantener una **precision razonable** para evitar reportar falsos positivos a las autoridades, lo cual podría generar confusión y desviar recursos. Y por último, queremos contar con un **buen balance entre precision y recall (F1-Score)** para asegurar que el modelo no optimice solo una métrica a costa de la otra, lo cual es especialmente importante en un contexto de desbalance de clases como el nuestro.

A la luz de los resultados obtenidos, el modelo cumple parcialmente con estos objetivos:

- **DESAPARECIDA (clase mayoritaria):** Obtuvimos un recall de **82.45%** y una precision muy alta de **98.99%** ($F1 = 89.97\%$). Aunque la precision es excelente (cuando el modelo predice DESAPARECIDA, casi siempre acierta), el recall es moderado: aproximadamente el 17.55% de los casos reales de DESAPARECIDA están siendo clasificados incorrectamente, en su mayoría como NO LOCALIZADA (16.07% según la matriz normalizada). Para una aplicación operativa real, sería deseable subir este recall por encima del 90%.
- **DESCONOCIDO (clase intermedia):** Recall perfecto de **100%** y precision de **97.61%** ($F1 = 98.79\%$). El modelo identifica todos los registros con información incompleta sin excepción, lo cual es coherente con que las variables más relevantes del modelo sean precisamente las categorías “_DESCONOCIDO”.
- **NO LOCALIZADA (clase minoritaria):** Recall alto de **85.79%** gracias a la costo-sensibilidad aplicada, pero la precision es muy baja: apenas **24.50%** ($F1 = 38.11\%$). Esto significa que de cada 4 casos que el modelo predice como NO LOCALIZADA, solo 1 lo es realmente. Los otros 3 son en su mayoría falsos positivos provenientes de

DESAPARECIDA. Este es el principal trade-off del modelo: priorizamos detectar casos resueltos (alto recall) a costa de generar falsas alarmas.

En resumen, el modelo logra un buen desempeño global (accuracy = **89.01%**, F1 ponderado = **91.33%**), pero con limitaciones claras: la precision en NO LOCALIZADA es baja por el desbalance extremo, y el recall en DESAPARECIDA podría mejorarse. Estas características deben ser consideradas si el modelo se llegara a aplicar en un contexto operativo real.

```
print("Análisis detallado de métricas por clase")

# Extraer valores de la matriz
tp_desap = cm[0, 0]
tp_descon = cm[1, 1]
tp_noloc = cm[2, 2]

print(f"\nDESAPARECIDA (Clase Mayoritaria):")
print(f" - Verdaderos Positivos: {tp_desap:,}")
print(f" - Precision: {precision_per_class[0]:.4f} ({precision_per_class[0]*100:.2f}%)")
print(f" - Recall: {recall_per_class[0]:.4f} ({recall_per_class[0]*100:.2f}%)")
print(f" - El modelo identifica correctamente la mayoría de casos activos")

print(f"\nDESCONOCIDO (Clase Intermedia):")
print(f" - Verdaderos Positivos: {tp_descon:,}")
print(f" - Precision: {precision_per_class[1]:.4f} ({precision_per_class[1]*100:.2f}%)")
print(f" - Recall: {recall_per_class[1]:.4f} ({recall_per_class[1]*100:.2f}%)")
print(f" - El modelo caracteriza adecuadamente registros incompletos")

print(f"\nNO LOCALIZADA (Clase Minoritaria):")
print(f" - Verdaderos Positivos: {tp_noloc:,}")
print(f" - Precision: {precision_per_class[2]:.4f} ({precision_per_class[2]*100:.2f}%)")
print(f" - Recall: {recall_per_class[2]:.4f} ({recall_per_class[2]*100:.2f}%)")
print(f" - La costo-sensibilidad permitió al modelo aprender patrones de casos resueltos")
```

Análisis detallado de métricas por clase

DESAPARECIDA (Clase Mayoritaria):

- Verdaderos Positivos: 13,173
- Precision: 0.9899 (98.99%)
- Recall: 0.8245 (82.45%)
- El modelo identifica correctamente la mayoría de casos activos

DESCONOCIDO (Clase Intermedia):

- Verdaderos Positivos: 9,830
- Precision: 0.9761 (97.61%)
- Recall: 1.0000 (100.00%)
- El modelo caracteriza adecuadamente registros incompletos

NO LOCALIZADA (Clase Minoritaria):

- Verdaderos Positivos: 833
- Precision: 0.2450 (24.50%)
- Recall: 0.8579 (85.79%)
- La costo-sensibilidad permitió al modelo aprender patrones de casos resueltos

6.14 Visualización de un Árbol del Random Forest

```
from sklearn import tree
import matplotlib.pyplot as plt

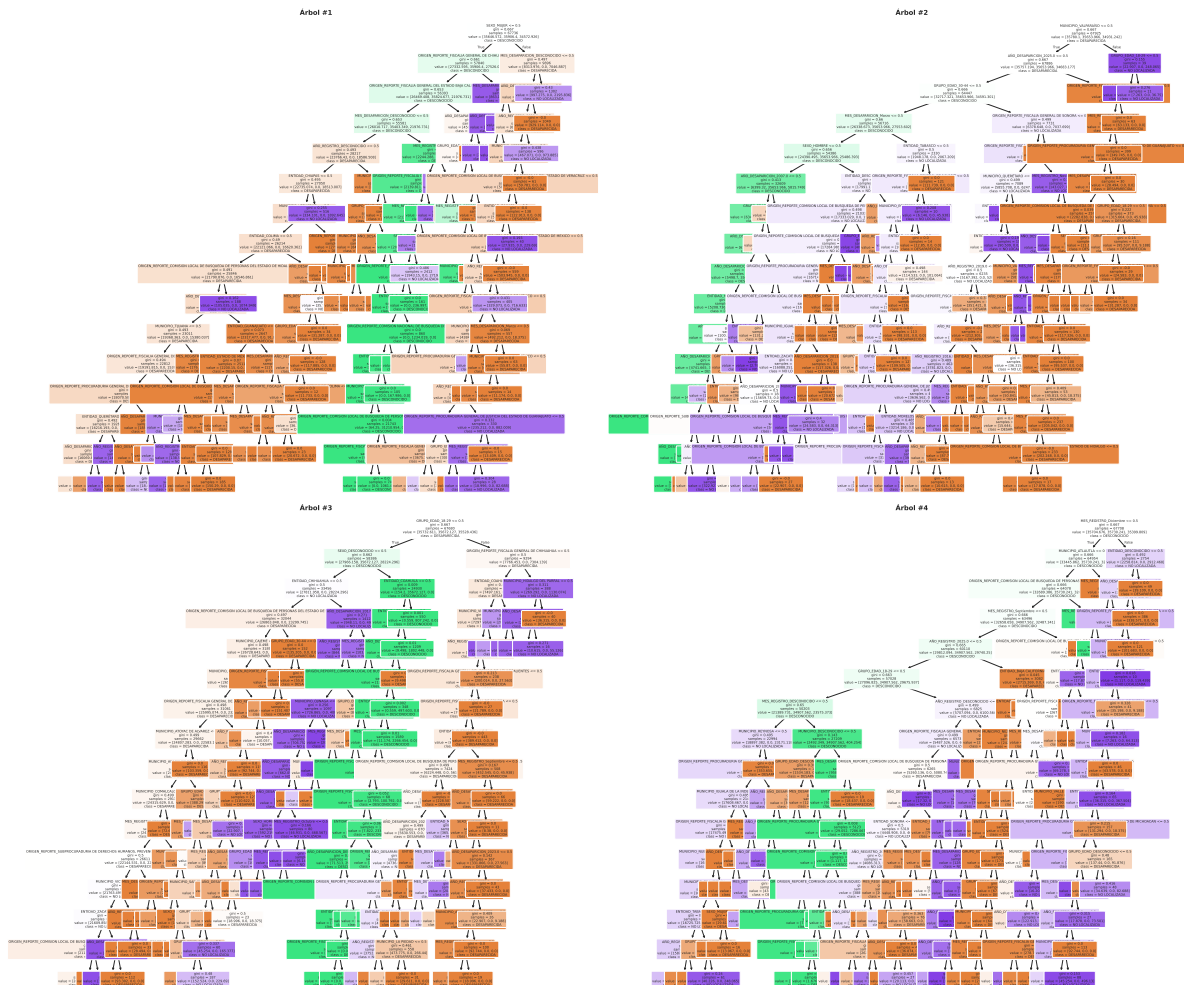
# Visualizar el primer árbol del ensemble
fig, ax = plt.subplots(figsize=(25, 15))
tree.plot_tree(
    rf_model.estimators_[0], # Primer árbol (índice 0)
    feature_names=X_encoded.columns.tolist(), # Nombres de variables
    class_names=le.classes_, # Nombres de clases
    filled=True, # Colorear nodos por clase mayoritaria
    rounded=True, # Esquinas redondeadas
    fontsize=8,
    ax=ax
)
plt.title("Árbol de Decisión #1 del Random Forest", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print(" Árbol 1 visualizado")
```



```
plt.tight_layout()
plt.show()

print(" Primeros 4 árboles visualizados")
```



Primeros 4 árboles visualizados

References